

NORTH LAKHIMPUR COLLEGE (AUTONOMOUS)

**CURRICULUM FRAMEWORK FOR THREE
YEAR UG DEGREE PROGRAMME (upto 6th
Semester)**

**Bachelor of Computer Application (BCA) and
MINOR, SEC and MDC for other programs**

**Department of Computer Science
(As per NEP 2020 Guidelines)**

(As approved by Board of Studies in Computer Science held on 01/06/2025 and passed by Academic Council held on)

Contents

1.0 Introduction:	1
1.1 Nature and extent of the B.C.A. program	1
1.2 Aims of bachelor's degree Programme in Computer Applications (BCA)	2
1.3 Graduate Attributes of Bachelor of Computer Applications (BCA) program	3
1.4 Qualification Descriptors for Bachelor of Computer Applications (BCA) program	4
1.5 Programme Specific Outcome	5
1.6 Course Outcomes for Bachelor of Computer Applications (BCA) program	6
1.7 Teaching Learning Methodologies	10
1.8 Assessment and Outcome Evaluation Methods	11
2.0 Certification Criteria	11
2.1 Other Key Criteria for UG Program	12
Summary of the UG Course Structure as per NEP Guidelines (upto 6 th Semester)	13
Structure of Syllabus	13
SEMESTER I	13
SEMESTER II	14
SEMESTER III	15
SEMESTER IV	16
SEMESTER V	17
SEMESTER VI	18
MAJ-5-CSC-101: Digital Design and Mathematical Foundation-I	19
MIN –5-CSC-101 Programming in C	24
SEC–3-CSC-101 Introduction to Web Design	28
MDC–3-CSC-101 Office Automation	31
MAJ –5-CSC-201: Computer Organization and Mathematical Foundation-II	34
MIN –5-CSC-201: Data Structure through C++	38
SEC –3-CSC-201: Dynamic Web Development	42
MDC –3-CSC-201: Working with Multimedia	45
MAJ –4-CSC-301: Operating Systems	49
MAJ –4-CSC-302: Database Management System	52
MIN –2-CSC-301: PHP Programming	55
MIN –2-CSC-302: Probability and Statistics	58
SEC –3-CSC-301: Advanced Web Design	61
MDC –3-CSC-301: Introduction to Financial Accounting Package	64
MAJ –4-CSC-401: Discrete Mathematics	67

MAJ –4-CSC-402: OOP with JAVA.....	70
MAJ –4-CSC-403: Theory of Computing	74
DSE –4-CSC-401: Introduction to Management and Financial Accounting.....	78
MAJ-5-CSC-501: Computer Networks	82
MAJ-4-CSC-502: Artificial Intelligence	86
MAJ-4-CSC-503: Software Engineering	90
DSE-3-CSC-501: Python Programming	93
MAJ-5-CSC-601: Project.....	97
MAJ-4-CSC-602: Computer Graphics.....	100
MAJ-4-CSC-603: Introduction to Machine Learning	104
MAJ-4-CSC-604: Design and Analysis of Algorithms	107
DSE-3-CSC-601: Android Progammig using JAVA/Cotlin.....	110

1.0 Introduction:

The Bachelor of Computer Applications (BCA) program is a dynamic and industry-relevant undergraduate degree designed to provide students with a strong foundation in computer science, software development, and information technology. This program equips students with essential technical skills, programming expertise, and computational thinking, preparing them for diverse careers in the ever-evolving tech industry.

Blending theoretical concepts with hands-on practical applications, the BCA curriculum covers software engineering, database management, web technologies, artificial intelligence, and cyber security. Students gain valuable experience through coding projects, and internships, enhancing their problem-solving abilities and technological proficiency.

Aligned with the New Education Policy (NEP) 2020, the program follows a multidisciplinary approach, integrating core computing principles with data-driven decision-making, digital innovation, and entrepreneurial skills. Emphasizing ICT applications, leadership, ethics, and sustainability, the BCA degree fosters critical thinking and adaptability in an era of rapid technological advancements.

With the increasing demand for IT professionals, BCA graduates can pursue careers in software development, data analytics, IT consulting, cybersecurity, and digital transformation. They can also opt for higher studies in computer science, artificial intelligence, or related IT fields to specialize in cutting-edge fields.

Ideal for students with a keen interest in technology, problem-solving, and software development, the BCA program serves as a gateway to a successful and future-ready career in the digital world.

1.1 Nature and extent of the B.C.A. program

The National Curriculum Framework for Undergraduate Studies in Computer Science is an outcome-based framework designed to establish academic standards and expected learning outcomes for graduates. This framework ensures that students acquiring a qualification in Computer Science develop the necessary technical knowledge, problem-solving skills, and computational competencies to meet both academic and professional demands.

The curriculum is structured around key components such as Graduate Attributes, Qualification Descriptors, Programme Learning Outcomes, and Course-Level Learning Outcomes. These elements serve as guiding principles for curriculum planning at the undergraduate level.

The overall objectives of the learning outcomes-based curriculum framework are to:

- Define and articulate graduate attributes, qualification descriptors, and learning outcomes at both program and course levels, ensuring clarity in academic expectations.

- Provide students, parents, employers, and stakeholders with a clear understanding of the technical skills, problem-solving abilities, ethical considerations, and professional values that a graduate is expected to demonstrate upon program completion.
- Maintain national academic standards while ensuring international comparability, enhancing the global competitiveness of graduates and supporting student mobility across institutions and countries.
- Assist higher education institutions in designing effective teaching-learning strategies, student assessment methods, and periodic curriculum reviews to maintain academic excellence and relevance in the rapidly evolving IT sector.

The BCA program integrates theoretical foundations, hands-on practical skills, and problem-solving expertise, equipping graduates for higher studies, research, and diverse career opportunities in software development, artificial intelligence, cloud computing, cybersecurity, and emerging digital technologies. By aligning with global standards, the program ensures that graduates remain technologically competent, adaptable, and industry-ready in an ever-evolving digital landscape.

1.2 Aims of bachelor's degree Programme in Computer Applications (BCA)

The Bachelor of Computer Applications (BCA) program aims to provide students with a strong foundation in computer science, software development, and information technology, equipping them with the necessary skills for academic and professional growth. The program is designed to meet national and international standards, ensuring graduates are well-prepared for diverse career opportunities and higher studies in emerging technology domains.

The key aims of the program are to:

- Develop a comprehensive understanding of computer science principles, including programming, software development, database management, networking, cybersecurity, and artificial intelligence.
- Foster critical thinking, problem-solving abilities, and analytical reasoning to enable students to address real-world technological challenges using computational tools and methodologies.
- Provide hands-on experience with modern programming languages, development frameworks, and IT infrastructure, enhancing students' ability to design, develop, and implement innovative software solutions.
- Promote the application of computing and IT solutions in various domains such as business, finance, healthcare, e-commerce, education, and smart technologies.
- Equip students with research skills and the ability to analyze, design, and develop software applications, incorporating project-based learning and industry internships.
- Align with the National Curriculum Framework and New Education Policy (NEP) 2020, ensuring a multidisciplinary, skill-based, and industry-oriented approach to learning.
- Instill ethical values, professional responsibility, and a commitment to lifelong learning, fostering a responsible approach to cybersecurity, data privacy, and technology ethics.
- Enhance communication, teamwork, leadership, and ICT skills, preparing students for diverse roles in software engineering, IT consulting, system administration, data analytics, cloud computing, and entrepreneurship.

- Ensure global competitiveness by maintaining academic excellence, facilitating international mobility, and enabling graduates to pursue advanced studies in computer science, artificial intelligence, machine learning, and emerging technologies.

By achieving these aims, the BCA program ensures that graduates are well-equipped to contribute to industry, research, and entrepreneurship, fostering innovation, digital transformation, and technology-driven solutions in the modern world.

1.3 Graduate Attributes of Bachelor of Computer Applications (BCA) program

The Bachelor of Computer Applications (BCA) program aims to develop well-rounded graduates with a strong foundation in computer science, software development, and emerging technologies. The graduate attributes define the key competencies, skills, and values that students are expected to demonstrate upon completion of the program.

- **Disciplinary Knowledge:** Graduates will have a comprehensive understanding of computer science principles, programming, algorithms, data structures, software engineering, and IT infrastructure, along with their real-world applications.
- **Analytical and Problem-Solving Skills:** Graduates will be able to analyze complex problems, develop algorithms, and implement efficient solutions using computational and logical reasoning in various domains.
- **Research and Innovation:** Students will develop a research-oriented mindset and the ability to explore, design, and innovate software solutions, AI-driven applications, and emerging technologies to address industry challenges.
- **Computational and ICT Proficiency:** Graduates will gain proficiency in programming languages, databases, cloud computing, artificial intelligence, cyber-security, and modern development frameworks, enabling them to build scalable and efficient IT solutions.
- **Critical Thinking and Logical Reasoning:** The program nurtures logical thinking, debugging skills, and system optimization techniques, ensuring students can critically evaluate technological problems and propose effective solutions.
- **Communication and Presentation Skills:** Graduates will develop the ability to effectively communicate technical concepts, write project reports, and present ideas to technical and non-technical audiences through various digital platforms.
- **Ethical Awareness and Professional Responsibility:** Students will be instilled with ethical principles in data security, privacy laws, intellectual property rights, and responsible AI, ensuring professional integrity and compliance with global IT standards.
- **Leadership, Teamwork, and Interpersonal Skills:** Graduates will demonstrate teamwork, collaboration, and leadership abilities, essential for working in multidisciplinary environments and managing software projects.
- **Lifelong Learning and Adaptability:** With rapidly evolving technological landscapes, students will develop a lifelong learning mindset, staying updated with the latest advancements in AI, cyber-security, blockchain, IoT, and cloud computing.

- **Global Competence and Societal Contribution:** Graduates will be equipped to apply computing knowledge for societal development, addressing real-world challenges in healthcare, smart cities, and sustainability, aligning with Sustainable Development Goals (SDGs).

By acquiring these attributes, graduates of the BCA program will be well-prepared for higher studies, research, industry roles, and entrepreneurial ventures in the ever-expanding field of technology and digital transformation.

1.4 Qualification Descriptors for Bachelor of Computer Applications (BCA) program

The Qualification Descriptors for the BCA outline the key competencies, skills, and knowledge that graduates are expected to demonstrate upon completion of the program. These descriptors ensure alignment with national and international academic standards, fostering academic excellence, employability, and research potential in the evolving field of technology.

- A. **Comprehensive Knowledge and Understanding:** Graduates will have a broad and in-depth understanding of computer science principles, programming paradigms, software engineering, database management, cloud computing, and artificial intelligence, along with their applications in various industries.
- B. **Analytical and Problem-Solving Skills:** Graduates will develop the ability to analyze complex computational problems, design algorithms, optimize systems, and develop efficient software solutions for real-world challenges in business, healthcare, finance, and smart technologies.
- C. **Research and Inquiry-Based Learning:** The program prepares students to conduct independent research, explore emerging technologies, innovate in AI and cybersecurity, and apply computational methodologies to advance the field of computer science.
- D. **Technical and Computational Proficiency:** Graduates will gain expertise in modern programming languages (Python, Java, C++, etc.), full-stack development, machine learning, cloud platforms, blockchain, and DevOps tools, enabling them to build and manage scalable IT solutions.
- E. **Communication and Technical Documentation:** Students will be able to effectively communicate technical concepts, document software solutions, and present findings through reports, presentations, and visual representations, catering to both technical and non-technical audiences.
- F. **Ethical and Social Responsibility:** Graduates will understand the ethical implications of technology, including cybersecurity, data privacy laws, AI ethics, and digital governance, ensuring the responsible use of technology for societal benefit.
- G. **Lifelong Learning and Adaptability:** The program fosters a lifelong learning mindset, enabling graduates to stay updated with emerging technologies, adapt to industry trends, and continuously upgrade their technical and managerial skills.

- H. **Employability and Career Readiness:** Graduates will be well-prepared for diverse career opportunities in software development, IT consulting, cybersecurity, cloud computing, AI/ML, fintech, and digital transformation, or for further academic pursuits in computer science, data analytics, and AI research.
- I. **Multidisciplinary and Global Perspectives:** The curriculum integrates multidisciplinary approaches and international academic standards, ensuring graduates possess a global outlook and the ability to work across industries, research institutions, and multinational organizations worldwide.

By achieving these qualification descriptors, graduates of the BCA program will be equipped with the expertise and skills required for professional success, research excellence, and technological innovation, making meaningful contributions to the digital economy and global technology landscape.

1.5 Programme Specific Outcome

Programme Specific outcomes relating to bachelor degree in Computer Science/Application are as follows:

- PSO 1. **Core Competency in Computer Science:** Demonstrate strong understanding of key computer science areas such as programming, algorithms, databases, software development, and emerging technologies.
- PSO 2. **Effective Communication Skills:** Develop proficiency in oral and written communication to clearly convey technical ideas to both technical and non-technical audiences.
- PSO 3. **Software Development & Problem Solving:** Apply CS knowledge and programming skills to design and build efficient, scalable, real-world software solutions.
- PSO 4. **Algorithm & System Optimization:** Analyze and implement algorithms and data structures, focusing on performance, memory, and resource optimization using software engineering principles.
- PSO 5. **Ethics and Social Responsibility:** Understand ethical, legal, and social issues in computing, promoting responsible and professional use of technology.
- PSO 6. **Data & Web Technologies:** Design and maintain data-driven applications and web services, ensuring data integrity, security, and accessibility.
- PSO 7. **Computational and Analytical Thinking:** Use mathematical, statistical, and computational methods to solve complex problems in AI, data analytics, and related fields.
- PSO 8. **Teamwork and Project Management:** Exhibit leadership and teamwork in managing IT projects, ensuring quality, timely, and cost-effective outcomes.
- PSO 9. **Lifelong Learning:** Pursue continuous learning and adapt to technological changes through professional development and self-improvement.
- PSO 10. **Professional Contribution:** Engage in community, open-source, and industry initiatives while upholding professional ethics and standards.

These outcomes help ensure that BCA graduates are equipped with the necessary technical, problem-solving, and professional skills to excel in the fast-evolving field of computer science.

1.6 Course Outcomes for Bachelor of Computer Applications (BCA) program

The Course Outcomes (COs) define the specific knowledge, skills, and abilities that students will acquire upon completing individual courses in the Bachelor of Computer Applications (BCA) program. These outcomes ensure alignment with the overall Programme Learning Outcomes (PLOs) and provide students with a structured learning path across different areas of Computer Science.

1. Convert between number systems and perform arithmetic operations on integers and floating-point numbers.
2. Learn to design combinational circuits using logic gates and simplify Boolean expressions.
3. Learn to design and analyze sequential circuits with flip-flops and timing diagrams.
4. Design and implement registers, counters, and shift registers.
5. Understand memory units and their organization in digital systems.
6. Matrix operations, calculate the inverse and determinant of matrices, and solve linear equations using matrices.
7. The concepts like divisibility, prime numbers, and modular arithmetic to solve problems.
8. Operations on complex numbers and express them in polar form.
9. Compute derivatives and higher-order derivatives, and apply them to real-world problems.
10. Master integration techniques and use them to find areas and solve related problems.
11. Demonstrate an understanding of the basic concepts of procedural programming
12. Use the C languages to write basic programs using variables, constants, operators and input/output statements.
13. Implement conditional statements and loops to control program flow.
14. Learn to create and use functions to modularize code and improve program structure.
15. Implement one-dimensional and multi-dimensional arrays to store and manipulate data.
16. Use structures and unions to create new data types and manipulate data.
17. Use pointers and references to manipulate data and memory allocation and use file input/output operations to read and write data from files.
18. Learn to create structured HTML documents, format text, and understand the core syntax and structure of HTML.
19. Implement internal and external hyperlinks, and insert and style images in web pages.
20. Learn to create and format ordered/unordered lists and tables to organize content effectively.
21. Learn to design functional forms, use appropriate form elements, and apply semantic HTML for better web accessibility.
22. Apply CSS styles to HTML elements, understand and use CSS selectors, and utilize the Box Model and layout techniques.

23. Learn to build responsive websites using media queries and the Bootstrap framework to ensure compatibility across different devices.
24. Demonstrate a clear understanding of Open Source Software and its benefits in office automation.
25. Learn to create and format documents, insert various objects, and effectively use Mail Merge in open-source word processing software.
26. Use open-source spreadsheet software to create and format spreadsheets, apply formulas, and create charts and graphs.
27. Learn to design and format presentations, and incorporate multimedia and animations to enhance presentation effectiveness.
28. Explain CPU and memory organization, instruction formats, and I/O operations..
29. Describe CPU registers, instruction cycles, and control unit types.
30. Explain cache, associative, and virtual memory organization.
31. Describe I/O methods including interrupts and DMA.
32. Summarize and describe data using measures of central tendency and dispersion.
33. Apply basic and conditional probability, including Bayes' theorem.
34. Compute expectation, variance, and moments of random variables.
35. Differentiate and use probability mass and density functions.
36. Calculate correlation and perform simple linear regression.
37. Explain fundamental OOP concepts and basic data structure terminology.
38. Implement and analyze arrays and various types of linked lists.
39. Apply stack and queue operations and solve problems using recursion.
40. Perform operations on binary search trees and balanced trees.
41. Implement and compare searching, sorting, and hashing algorithms.
42. Describe the web development process and common tools used.
43. Write JavaScript code using variables, control statements, functions, and manipulate the DOM.
44. Implement AJAX and JSON techniques and use JavaScript libraries effectively.
45. Create dynamic web pages using DHTML, CSS styling, and XML parsing.
46. Explain multimedia types and the significance of open-source software in multimedia.
47. Design and optimize graphics using open-source tools like GIMP and Inkscape.
48. Record, edit, and manage audio using software such as Audacity and Ardour.
49. Capture and edit videos using open-source video production tools.
50. Create 2D and 3D animations using Synfig and Blender.
51. Describe the core functions and types of operating systems and their evolution.
52. Explain process/thread management, scheduling algorithms, and deadlock concepts
53. Demonstrate memory allocation methods, file operations, and devicemanagement techniques.
54. Apply synchronization mechanisms and inter-process communication methods.
55. Illustrate access control, security measures, and basic concepts of distributed systems.
56. Explain the importance, types, and architecture of database systems.
57. Create ER diagrams and apply relational model concepts for data modeling.
58. Construct SQL queries and perform database normalization.
59. Demonstrate understanding of transactions, concurrency control, and recovery techniques.
60. Implement database recovery procedures and apply security mechanisms.
61. Explain PHP basics and demonstrate use of variables, data types, and operators.

62. Write PHP scripts using conditional statements, loops, and user-defined functions.
63. Develop PHP applications that interact with MySQL databases for data manipulation.
64. Handle HTML form submissions and implement session management in PHP.
65. Explain and identify different types of probability distributions such as binomial, Poisson, and normal.
66. Differentiate between sampling techniques and apply appropriate methods for selecting samples.
67. Demonstrate how sampling distributions work and use the Central Limit Theorem in estimation problems.
68. Formulate hypotheses and interpret the implications of Type I and Type II errors in hypothesis testing.
69. Describe the roles of clients, web servers, and databases and how they interact in the web stack.
70. Apply advanced HTML and CSS techniques and use template engines to create dynamic web pages.
71. Demonstrate knowledge of deploying websites manually and through automated CI/CD pipelines.
72. Implement RESTful APIs with CRUD operations in web applications.
73. Integrate external APIs into web applications for enhanced functionality.
74. Explain key financial accounting principles and complete the accounting cycle including preparation of trial balances.
75. Demonstrate effective use of accounting software to set up company profiles and navigate its features.
76. Accurately enter and post sales, purchases, expenses, and payments to relevant accounts.
77. Generate and interpret financial statements such as balance sheets, income statements, and cash flow reports using accounting software.
78. Apply set theory, relations, functions, and combinatorics to solve discrete mathematical problems.
79. Illustrate the structure and properties of algebraic systems including groups and rings.
80. Analyze and interpret graphs and trees, determine connectivity, and apply graph traversal algorithms.
81. Construct and evaluate logical statements using propositional logic, tautologies, and inference rules.
82. Apply Java syntax and control structures to write basic Java programs.
83. Manipulate arrays and string objects efficiently using built-in classes and methods.
84. Implement object-oriented programming principles like encapsulation, method overloading, and inner classes.
85. Demonstrate inheritance, method overriding, and the use of interfaces and abstract classes.
86. Build Java programs with proper exception handling, multithreading, and file I/O operations.
87. Design and analyze finite automata (DFA, NFA, ϵ -NFA) and convert between them.
88. Classify languages using the Chomsky hierarchy and explain the relationships among different types of grammars.

89. Construct regular expressions and regular grammars for formal languages, and prove non-regularity using the pumping lemma.
90. Construct context-free grammars and design pushdown automata for context-free languages, including transformations between them.
91. Explain the structure and behavior of Turing machines and demonstrate understanding of decidability and the halting problem.
92. Explain the meaning, purpose, and scope of accounting and record basic accounting transactions using journal entries and ledgers.
93. Prepare trial balances, trading and profit & loss accounts, and balance sheets, and apply accounting treatment for depreciation.
94. Analyze financial management functions, determine cost of capital, and evaluate working capital requirements.
95. Prepare different types of budgets and apply capital budgeting techniques and marginal costing concepts in decision-making.
96. Explain the structure and functioning of computer networks, compare OSI and TCP/IP models, identify network topologies, and describe the roles of common networking devices.
97. Analyze data link layer functions, implement subnetting, understand MAC addressing, and configure basic VLANs and ARP settings.
98. Differentiate between routing protocols and transport layer protocols, explain congestion control strategies, and apply basic network security concepts such as firewalls and VPNs.
- 99.** Describe the operation of application layer protocols, evaluate the role of technologies like SDN, IoT, and 5G, and apply tools for monitoring and managing network performance.
100. Identify and explain the concepts of AI, intelligent agents, and their environments.
101. Implement and evaluate various search algorithms like DFS, BFS, A*, and genetic algorithms.
102. Apply logical reasoning techniques and handle uncertainties using probabilistic and fuzzy set theory.
103. Understand and apply AI techniques in real-world domains such as NLP, robotics, and expert systems.
104. Apply various software development models, including agile and traditional models, to real-world projects.
105. Demonstrate the ability to collect, analyze, and validate software requirements, ensuring the development of quality software.
106. Design software architectures, create appropriate diagrams, and apply testing strategies for quality assurance.
107. Effectively manage project risks, plan releases, and maintain software products through their lifecycle.
108. Demonstrate proficiency in Python syntax, data types, control flow, and functions.
109. Apply concepts of mutable and immutable objects, including lists, tuples, and dictionaries, and use libraries like Numpy for data manipulation.
110. Manage files, handle exceptions, and create basic data visualizations using Python.
111. By the end of the BCA Project, students will be able to design, develop, test, and document a real-world software application by applying theoretical and practical knowledge of computer science, demonstrating proficiency in system analysis,

- software development life cycle (SDLC), user interface design, coding, testing strategies, and deployment, while adhering to professional standards of documentation, teamwork, and project presentation.
112. Explain the architecture and components of computer graphics systems, including display and I/O devices.
 113. Implement raster algorithms and clipping techniques for 2D graphics primitives.
 114. Apply geometric and viewing transformations to 2D and 3D graphical objects, including projection techniques.
 115. Model 3D shapes using polygon meshes and represent curves using Hermite and Bézier methods.
 116. Apply algorithms for visible surface determination, describe shading and illumination models, and explain basic animation concepts.
 117. Explain types of machine learning, differentiate between regression and classification, and evaluate models using standard performance metrics.
 118. Apply ML algorithms like regression, classification, and clustering, and analyze case studies while understanding ethical implications.
 119. Analyze basic algorithms using asymptotic notations and apply complexity analysis techniques to recursive and iterative algorithms.
 120. Implement and evaluate divide & conquer and greedy algorithms such as merge sort, quick sort, Kruskal's, and Dijkstra's algorithms.
 121. Solve optimization problems using dynamic programming and apply graph traversal techniques like BFS and DFS for various applications.
 122. Implement backtracking algorithms and classify problems based on computational complexity into P, NP, NP-Complete, and NP-Hard categories.
 123. Describe Android architecture and development tools, and build a basic Android application using Android Studio.
 124. Design and implement user interfaces using layouts, views, themes, and handle various user events.
 125. Develop applications using multiple activities, fragments, and intents for seamless navigation and modular design.
 126. Integrate multimedia features, send SMS and emails, and implement location-based services using Google Maps and notifications.
 127. Manage local storage using SQLite and SharedPreferences, implement HTTP networking using JSON, and build, test, and publish Android apps to the Play Store.

These Course Outcomes (COs) ensure that students develop a strong theoretical foundation, practical skills, and research capabilities across various Computer Science/IT domains, preparing them for academia, industry, and research careers.

1.7 Teaching Learning Methodologies

The BCA program is structured to facilitate the development of subject-specific knowledge, technical expertise, and both academic and professional skills essential for careers in software development, IT services, and related fields. Learning experiences are designed to promote active engagement, hands-on practice, and participatory learning to equip students with practical problem-solving abilities.

Instructional methods will include lectures, lab demonstrations, coding exercises, group discussions, hands-on projects, presentations, peer programming, quizzes, workshops,

hackathons, technical seminars, tutorials, and experiential learning. Students will be encouraged to explore real-world IT challenges and engage with industry-driven projects and internships. To support this, practical components are integrated into multiple courses.

The use of ICT tools, online coding platforms, cloud-based resources, and web-based learning materials is strongly encouraged to enhance interactivity, engagement, and real-world applicability in the teaching-learning process.

1.8 Assessment and Outcome Evaluation Methods

The evaluation of students' achievement in the BCA program will be aligned with the course and program learning outcomes, as well as the technical, analytical, and professional skills the program aims to develop. A diverse range of assessment methods, appropriate to the field of Computer Science and Information Technology, will be utilized.

Learning outcomes will be assessed through written and oral examinations, closed-book and open-book tests, coding assignments, algorithmic problem-solving exercises, database and software development projects, programming tasks, laboratory reports, individual and group project reports, technical presentations, viva voce interviews, literature reviews, case studies, research projects, hackathons, and internship evaluations.

2.0 Certification Criteria

- **UG Certificate:** Students exiting the programme after securing 40 credits will be awarded UG Certificate in the relevant discipline/subject provided they secure 4 credits in work-based vocational courses offered during the summer term or internship/apprenticeship in addition to 6 credits from skill-based courses earned during the first and second semester.
- **UG Diploma:** Students exiting the programme after securing 80 credits will be awarded UG Diploma in the relevant discipline/subject.
- **3-year UG Degree:** Students who wish to undergo a 3-year UG programme will be awarded UG Degree in the Major discipline after successful completion of three years, securing 120 credits.
- **4-year UG Degree (Honours):** A four-year UG Honours degree in the major discipline will be awarded to those who complete a four-year degree programme with 160 credits.
- **4-year UG Degree (Honours with Research):** A four-year UG Degree in Honours with Research will be awarded to those Students who complete a four-year degree programme with 160 credits, including 12 credits from a research project/dissertation in the major discipline.

2.1 Other Key Criteria for UG Program

- **Summer Internship/Apprenticeship:** A summer internship or apprenticeship is mandatory for all Education students across all UG certification and degree levels, including the UG Certificate, UG Diploma, 3-year UG Degree.
 - For the **UG Certificate**, the internship must be completed during the summer break following the 2nd semester.
 - For the **UG Diploma**, it must be completed either after the 2nd or 4th semester.
 - For all other degree levels, it must be completed by the end of the 5th semester.

Students will choose a vocational course for their internship from the available options within the common program structure.
- **Exit and Re-entry:** Students enrolled in the Education degree program will have the flexibility to exit and re-enter at specific points. Those awarded a **UG Certificate** can exit after the 2nd semester (first year), while those receiving a **UG Diploma** can exit after the 4th semester (second year). Re-entry into the program will also be permitted at these stages.

Summary of the UG Course Structure as per NEP Guidelines (upto 6th Semester)

Department of Computer Science

Structure of Syllabus

SEMESTER I

Course	Course Title	Course Code	Credit	Marks	End Sem Marks	Internal Assessment
1	Digital Design and Mathematical Foundation-I	MAJ-5-CSC-101	5	100	70	30
2	Programming in C	MIN-5-CSC-101	3+2=5	100	42+40=82	18
3	Introduction to Web Design	SEC-3-CSC-101	1+2=3	80	28+40	12
4	AECC (Communicative English)		2	40	28	12
5	MDC To be chosen from other department		3			
6	VAC (IKS)		2	40	28	12
7	Office Automation (MDC)	MDC-3-CSC-101	1+2	80	28+40	12
- For the BCA programme, course serial numbers 1 to 6 are required. - Course serial numbers 3 and 7 are offered to students from other departments.						

SEMESTER II

Course	Course Title	Course Code	Credit	Marks	End Sem Marks	Internal Assessment
8	Computer Organization and Mathematical Foundation-II	MAJ-5-CSC-201	5	100	70	30
9	Data Structure using C++	MIN-5-CSC-201	3+2=5	100	42+40=82	18
10	Dynamic web Development	SEC-3-CSC-201	1+2=3	80	28+40	12
11	AECC (Language and Literature (MIL/Regional Language))		2	40	28	12
12	MDC To be chosen from other department		3			
13	VAC To be chosen from other department		2	40	28	12
14	Working with Multimedia (MDC)	MDC-3-CSC-201	1+2	80	28+40	12
- For the BCA programme, course serial numbers 8 to 13 are required. - Course serial numbers 9 and 14 are offered to students from other departments.						

SEMESTER III

Course	Course Title	Course Code	Credit	Marks	End Sem Marks	Internal Assessment
15	Operating System	MAJ-4-CSC-301	3+1=4	80	42+20	18
16	Database Management Systems	MAJ-4-CSC-302	3+1=4	80	42+20	18
17	PHP Programming	MIN-2-CSC-301	2	40	40	0
18	Probability and Statistics	DSE-2-CSC-301	2	40	28	12
19	Advanced Web Design	SEC-3-CSC-301	1+2	80	28+40	12
20	Critical Reading/Writing Skill (English)		2	40	28	12
21	MDC To be chosen from other department		3			
22	Introduction to Financial Accounting Package	MDC-3-CSC-301	1+2	80	28+40	12
- For the BCA programme, course serial numbers 15 to 21 are required. - Course serial numbers 17 and 22 are offered to students from other departments.						

SEMESTER IV

Course	Course Title	Course Code	Credit	Marks	End Sem Marks	Internal Assessment
23	Discrete Mathematics	MAJ-4-CSC-401	4	80	56	24
24	OOP with JAVA	MAJ-4-CSC-402	3+1=4	80	42+20	18
25	Theory of Computing	MAJ-4-CSC-403	4	80	56	24
26	Introduction to Management and Financial Accounting	DSE-4-CSC-401	3+1=4	80	42+20	18
27	Critical Reading/Writing Skill/ Book Reading (MIL/Regional Language)		2	40	28	12
28	VAC To be chosen from other department		2	40	28	12
For the BCA programme, all the courses are required.						

SEMESTER V

Course	Course Title	Course Code	Credit	Marks	End Sem Marks	Internal Assessment
29	Computer Networks	MAJ-5-CSC-501	3+2=5	100	42+40=82	18
30	Artificial Intelligence	MAJ-4-CSC-502	3+1=4	80	42+20	18
31	Software Engineering	MAJ-4-CSC-503	4	80	56	24
32	Programming with Python	DSE-3-CSC-501	2+1=3	60	28+20	12
33	Internship		4	80		
For the BCA programme, all the courses are required.						

SEMESTER VI

Course	Course Title	Course Code	Credit	Marks	End Sem Marks	Internal Assessment
34	Project	MAJ-5-CSC-601	5	100	70	30
35	Computer Graphics	MAJ-4-CSC-602	4	80	56	24
36	Introduction to Machine Learning	MAJ-4-CSC-603	2+2=4	80	28+40	18
37	Design and analysis of Algorithms	MAJ-4-CSC-604	4	80	56	24
38	Android Programming with JAVA	DSE-3-CSC-601	2+1=3	60	28+20	12
For the BCA programme, all the courses are required.						

Semester-I

MAJ-5-CSC-101: Digital Design and Mathematical Foundation-I

Course Code	: MAJ-5-CSC-101
Course Title	: Digital Design and Mathematical Foundation-I
Nature of the Course	: MAJOR
Total Credit assigned	: 5
Component of the Course	: Lecture: 4 Practical: 0 Tutorial: 1
Distribution of Marks	: End Semester: 70 In Semester: 30

Course Objectives

The main objectives of this course are:

- CO-1. Understand and work with number systems (binary, decimal, octal, hexadecimal), perform conversions, and work with signed/unsigned integers and floating-point representations.
- CO-2. Apply digital logic design principles, including Boolean algebra, logic gates, combinational circuits, and simplification using Karnaugh Maps.
- CO-3. Analyze and design sequential circuits using flip-flops, timing diagrams, and state tables.
- CO-4. Learn to design registers and counters (synchronous/asynchronous) and understand shift register applications.
- CO-5. Understand memory units (RAM and ROM) and their organization in digital systems.
- CO-6. Understand matrix operations, properties, and learn to solve systems of linear equations using matrices.
- CO-7. Grasp the fundamental concepts of number theory, including divisibility, prime numbers, the fundamental theorem of arithmetic, and modular arithmetic.
- CO-8. Learn the properties and operations of complex numbers, including their polar form.
- CO-9. Understand limits, continuity, derivatives, and their applications, along with higher-order derivatives.
- CO-10. Learn the basics of integration, integration techniques, and how to calculate areas using integrals.

Course Content

Section A: Digital Design

Unit	Content	Marks	L	P	T
I	Number Systems • Overview of number systems: binary, decimal, octal, and hexadecimal • Converting between number systems • Understanding signed and unsigned integer	11	7	0	3

	- representations - Two's complement representation for signed integers - Arithmetic operations on signed and unsigned integers - Fixed-point representation for fractional numbers - IEEE 754 standard for floating-point representation - Arithmetic operations on fixed and floating-point numbers - ASCII and Unicode encoding schemes - Representing characters and symbols				
II	Introduction to Digital Logic Design - Basic concepts of digital logic design - Boolean algebra principles and logic gates - Logic gate symbols, truth tables, and Boolean expressions - Designing combinational circuits using logic gates - Simplifying Boolean expressions using Karnaugh Maps (K-Maps) - Multiplexers and demultiplexers	13	8	0	2
III	Sequential Logic Circuits - Flip-flops and their types: SR, D, JK, T flip-flops - Designing and analyzing sequential circuits - Timing diagrams and state tables	6	5	0	1
IV	Registers and Counters - Introduction to registers and counters - Designing synchronous and asynchronous counters - Shift registers and their applications	5	5	0	1
V	Memory Units - Introduction to memory units in digital systems - RAM (Random Access Memory) and ROM (Read-Only Memory) - Memory organization	5	5	0	1
Total		40	30	0	8

Section B: Mathematical Foundation-I (Marks: 30)

Unit	Content	Marks	L	P	T
VI	Matrices - Introduction to matrices, matrix operations, and properties - Inverse and determinant of a square matrix - Solving systems of linear equations using matrices	6	9	0	2
VII	Number Theory Fundamental concepts of number theory	6	6	0	2

	• Divisibility, Prime numbers, • Fundamental theorem of arithmetic, modular arithmetic				
VIII	Complex Numbers • Introduction to complex numbers, their properties, and operations • The polar form of complex numbers	6	5	0	1
IX	Differential Calculus • Limits and continuity • Derivatives and their applications • Higher-order derivatives	6	5	0	1
X	Integral Calculus • Introduction to integration • Integration techniques • finding area	6	5	0	1
		30	30	0	7

Mode of In-Semester assessment (30 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

A. Sessional activity:

- ☐ Quizzes& Assignments
 - Regular quizzes on number systems, Boolean algebra, and sequential circuits.
 - Written assignments on Karnaugh Maps (K-Maps), logic simplifications, and memory units.
- ☐ Presentations& Case Studies
 - Group presentations on real-world applications of digital design, such as IEEE 754 floating-point representation, memory units, and logic gates in computers.
 - Case study on Boolean algebra and its use in circuit optimization.
- ☐ Viva& Problem-Solving Sessions
 - Oral viva on key topics like sequential circuits, memory organization, and Boolean algebra.
 - Problem-solving sessions focusing on matrix operations, differentiation, and integration techniques relevant to digital systems.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Convert between number systems and perform arithmetic operations on integers and floating-point numbers.
- CLO-2. Learn to design combinational circuits using logic gates and simplify Boolean expressions.
- CLO-3. Learn to design and analyze sequential circuits with flip-flops and timing diagrams.
- CLO-4. Design and implement registers, counters, and shift registers.
- CLO-5. Understand memory units and their organization in digital systems.

- CLO-6. Matrix operations, calculate the inverse and determinant of matrices, and solve linear equations using matrices.
- CLO-7. The concepts like divisibility, prime numbers, and modular arithmetic to solve problems.
- CLO-8. Operations on complex numbers and express them in polar form.
- CLO-9. Compute derivatives and higher-order derivatives, and apply them to real-world problems.
- CLO-10. Master integration techniques and use them to find areas and solve related problems.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₅ ,	CO ₁ , CO ₅ ,	CO ₁ ,			
Conceptual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₆ , CO ₇ , CO ₈ , CO ₉ , CO ₁₀	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₆ , CO ₇ , CO ₈ , CO ₉ , CO ₁₀	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₆ , CO ₇ , CO ₈ , CO ₉ , CO ₁₀			
Procedural Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₇ , CO ₈ , CO ₉ , CO ₁₀	CO ₁ , CO ₂ , CO ₃ , CO ₇ , CO ₈ , CO ₉ , CO ₁₀	CO ₁ , CO ₂ , CO ₃ , CO ₇ , CO ₈ , CO ₉ , CO ₁₀			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	2		1			1			
CLO2	3	2		1						
CLO3	3	2		1				1		
CLO4	3	2		1				1		
CLO5	3	2		1	1					
CLO6	2	2		1					2	
CLO7	2	3		1					2	1
CLO8	2	2		1						1
CLO9	2	3		1					2	1
CLO10	2	2		1	1	2	1		2	1
Average Contribution	2.5	2.4		1.0	0.2	0.2	0.2	0.2	0.8	0.4

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Mano, M. M. (1993). *Computer system architecture*. Prentice-Hall, Inc..
2. Stallings, W. (2016). *Computer organization and architecture: designing for performance*. Pearson Education India.
3. Mano, M. M. (2017). *Digital logic and computer design*. Pearson Education India.
4. Kreyszig, E. (2007). *Advanced engineering mathematics 9th edition* (p. 1094). John Wiley & Sons.
5. Grewal, B. S., &Grewal, J. S. (1996). *Higher engineering mathematics. 2002*, Khanna Publishers, New Delhi.
6. Lehman, E., Leighton, F. T., & Meyer, A. R. (2010). *Mathematics for computer science*. Cambridge, Massachusetts: Massachusetts Institute of Technology.

Semester-I

MIN –5-CSC-101 Programming in C

Course Code	: MIN –5-CSC-101
Course Title	: Programming in C
Nature of the Course	: MINOR
Total Credit assigned	: 5
Component of the Course	: Lecture: 3 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 42 Practical: 40 In Semester: 18

Course Objectives

The main objectives of this course are:

- CO-1. To introduce students to C programming, including its history, structure, and compilation process.
- CO-2. Learn to develop an understanding of data types, variables, operators, and input/output operations in C.
- CO-3. To equip students with skills to implement conditional and iterative statements for decision-making and loops.
- CO-4. To provide knowledge of functions, arrays, and string manipulation for efficient programming.
- CO-5. To introduce structures, unions, and pointers for handling complex data types.
- CO-6. To explain memory management techniques, including dynamic memory allocation and to familiarize students with file handling operations and pre-processor directives in C programming.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to C • Overview of programming languages • History of C • Basic structure of a C program • Compilation process • Distinction between C • Data types and variables • Constants and literals • Operators and expressions • Basic input/output operations • Formatted I/O	5	7	12	0
II	Expressions, Conditional Statements and Iterative Statement • Conditional statements (if-else, switch) • Iterative statements (while, do-while, for) • Nested loops and conditional statements	9	9	12	0

III	Functions and Arrays • Introduction to functions • Function prototypes • Call by value and call by reference • Return values and void functions • Recursive functions • One-dimensional arrays • Multi-dimensional arrays • Strings and string manipulation	10	13	12	0
IV	Derived Data Types (Structures and Unions) • Structures • Unions • Pointers to structures and unions • Arrays of structures and unions	6	6	8	0
V	Pointers and References • Introduction to pointers • Pointers to basic data types • Pointers to arrays and structures • Pointers vs. references	6	5	8	0
VI	Memory Allocation and File I/O • Static memory allocation • Dynamic memory allocation • File input/output operation	6	5	8	0
		42	45	60	0

Mode of In-Semester assessment (18 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

- ☐ Quizzes& Assignments
 - Weekly quizzes on C syntax, data types, control structures, and functions.
 - Assignments on arrays, pointers, memory allocation, and file handling.
- ☐ Viva& Problem Solving Sessions
 - Oral viva covering concepts of functions, recursion, and dynamic memory allocation.
 - Problem-solving discussions on complex programming scenarios and debugging techniques.

Lab Assignment:

1. Write a program to calculate the sum and product of digits of an integer number.
2. Write a program to reverse a number.
3. Write a program to compute the sum of the first n terms of the series $S = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots$

4. Write a program to compute the sum of the first n terms of the series $S = 1 - 2 + 3 - 4 + 5 \dots$
5. Write a program that checks whether a given string is Palindrome or not.
6. Write a program to find whether a given no. is prime or not. Use the same to generate the prime numbers less than 100.
7. Write a program to compute the factors of a given number.
8. Write a program to swap two numbers using macros.
9. Write a program to perform various operations on an array entered by the user.
10. Write a program to print a table indicating the number of occurrences of each alphabet in the text entered as command line arguments.
11. Write a program to swap two numbers using pointers.
12. Write a program in which a function is passed address of two variables and then alter its contents.
13. Write a program which takes the radius of a circle as input from the user and computes its area and circumference.
14. Write a program to find sum of n elements entered by the user using dynamic memory allocation.
15. Write a program to perform various operations on strings using menu-driven approach.
16. Write a program to merge two ordered arrays of integers to get an ordered array.
17. Write a program to display Fibonacci series using recursion and iteration.
18. Write a program to calculate Factorial of a number using recursion and iteration.
19. Write a program to calculate GCD of two numbers using recursion and iteration.
20. Write a program to implement Matrix operations using templates.
21. Write a program to create structure of 10 students and store them in a file.
22. Write a program to retrieve the student information from file and display it.
23. Write a program to copy contents of one text file to another after removing whites

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Demonstrate an understanding of the basic concepts of procedural programming
- CLO-2. Use the C languages to write basic programs using variables, constants, operators and input/output statements.
- CLO-3. Implement conditional statements and loops to control program flow.
- CLO-4. Learn to create and use functions to modularize code and improve program structure.
- CLO-5. Implement one-dimensional and multi-dimensional arrays to store and manipulate data.
- CLO-6. Use structures and unions to create new data types and manipulate data.
- CLO-7. Use pointers and references to manipulate data and memory allocation and use file input/output operations to read and write data from files.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂	CO ₁ , CO ₂				
Conceptual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆ , CO ₇	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆ , CO ₇	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆ , CO ₇			
Procedural Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆ , CO ₇	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆ , CO ₇	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆ , CO ₇			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	2	2			2	1	1	1
CLO2	3	1	2	2			2	1	1	1
CLO3	3	1	3	2			2	1	1	1
CLO4	3	1	3	2			2	1	1	1
CLO5	3	1	3	2			2	1	1	1
CLO6	3	1	2	2			2	1	1	1
CLO7	3	1	3	2	1		2	1	1	1
Average Contribution	3	1.0	2.57	2.0	0.14		0.57	0.42	1.0	1.0

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Kanetkar, Y. (2018). *Let us C*. BPB publications.
2. Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language*. prentice-Hall.
3. Griffiths, D., & Griffiths, D. (2012). *Head First C: A Brain-Friendly Guide*. " O'Reilly Media, Inc."
4. Balagurusamy, E. (2016). *programming in ANSI C*.

Semester-I

SEC-3-CSC-101 Introduction to Web Design

Course Code	: SEC-3-CSC-101
Course Title	: Introduction to Web Design
Nature of the Course	: SEC
Total Credit assigned	: 3
Component of the Course	: Lecture: 1 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 40 In Semester: 12

Course Objectives

The main objectives of this course are:

- CO-1. Understand the basics of HTML, including syntax, document structure, and text formatting.
- CO-2. Learn how to create hyperlinks, link to internal and external pages, and insert and format images.
- CO-3. Learn to create ordered and unordered lists, as well as design and format tables.
- CO-4. Understand how to create forms, use form elements, apply semantic markup, and ensure web accessibility.
- CO-5. Understand CSS syntax, style HTML elements, use CSS selectors, and explore the CSS Box Model, layouts, and positioning.
- CO-6. Learn the principles of responsive design, media queries, and use frameworks like Bootstrap for creating responsive websites.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to HTML • Introduction to Markup Languages • HTML syntax • Document structure • Text formatting	2	3	12	
II	Hyperlinks and Images • Creating hyperlinks • Linking to internal and external pages • Inserting images, Image formatting	4	3	8	
III	Lists and Tables • Creating lists • Ordered and unordered lists • Creating tables • Table formatting	5	1	8	
IV	Forms and Semantic Markup	4	2	8	

	• Creating forms • Form elements • Semantic markup • Web accessibility				
V	Introduction to CSS • CSS syntax • Applying styles to HTML elements • CSS selectors • CSS Box Model • Layouts and positioning • Creating columns • Creating menus	9	4	12	
VI	Responsive Web Design • What is responsive design? • Media queries • Responsive images • Bootstrap Framework	4	2	12	
		28	15	60	

Mode of In-Semester assessment (12 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

- ☐ Quizzes& Assignments
 - Weekly quizzes on HTML tags, CSS properties, and responsive design concepts.
 - Assignments on creating structured web pages using HTML and styling them with CSS.
- ☐ Viva& Problem-Solving Sessions
 - Oral viva covering HTML structure, CSS styling techniques, and responsive design principles.
 - Problem-solving discussions on layout techniques and debugging common HTML/CSS issues.

LabWork:

In the lab component, students will have hands-on experience creating and designing web pages using HTML and CSS. They will have access to computers with necessary software and internet connectivity. Lab exercises will cover HTML syntax, formatting, hyperlinks, images, lists, tables, forms, and semantic markup. Students will also learn how to use CSS to style web pages and create responsive designs. The final project will give students the opportunity to create a fully functional website.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Learn to create structured HTML documents, format text, and understand the core syntax and structure of HTML.

- CLO-2. Implement internal and external hyperlinks, and insert and style images in web pages.
- CLO-3. Learn to create and format ordered/unordered lists and tables to organize content effectively.
- CLO-4. Learn to design functional forms, use appropriate form elements, and apply semantic HTML for better web accessibility.
- CLO-5. Apply CSS styles to HTML elements, understand and use CSS selectors, and utilize the Box Model and layout techniques.
- CLO-6. Learn to build responsive websites using media queries and the Bootstrap framework to ensure compatibility across different devices.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁				
Conceptual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆			
Procedural Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	2	2		3	2	2	2	2
CLO2	3	1	2	2		3	2	2	2	2
CLO3	2	1	2	2		3	1	2	2	2
CLO4	2	1	2	2	1	3	2	2	2	2
CLO5	3	1	2	2	1	3	2	3	2	2
CLO6	3	1	2	2	1	3	2	3	2	2
Average Contribution	2.67	1	2	2	0.5	3.0	1.83	2.33	2	2

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Duckett, J. (2011). *Beginning html, xhtml, css, and javascript*. John Wiley & Sons.
2. Robbins, J. N. (2012). *Learning web design: A beginner's guide to HTML, CSS, JavaScript, and web graphics*. " O'Reilly Media, Inc."
3. McFarland, D. S. (2012). *CSS3: the missing manual*. " O'Reilly Media, Inc."

Semester-I

MDC-3-CSC-101 Office Automation

Course Code	: MDC-3-CSC-101
Course Title	: Office Automation
Nature of the Course	: MDC
Total Credit assigned	: 3
Component of the Course	: Lecture: 1 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 40 In Semester: 12

Course Objectives

The main objectives of this course are:

- CO-1. Understand the concept of Open Source Software (OSS), its advantages for office automation, and explore examples of OSS used in office automation tasks.
- CO-2. Learn to create, edit, and format text documents, insert tables, pictures, and objects, and use Mail Merge functionality in open-source word processing software.
- CO-3. Gain proficiency in using open-source spreadsheet software for creating, editing, formatting spreadsheets, applying formulas and functions, and creating charts and graphs.
- CO-4. Learn how to create, edit, and format presentations, and enhance them by adding multimedia and animations using open-source presentation software.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to Open Source Software for Office Automation • Definition of Open Source Software • Advantages of using Open Source Software for office automation • Examples of Open Source Software for office automation	4	3	16	0
II	Word Processing using Open Source Software • Introduction to Open Source Software for word processing • Creating, editing and formatting text documents • Inserting tables, pictures, and other objects in documents • Mail Merge	8	4	16	0
III	Spreadsheet using Open Source Software • Introduction to Open Source Software for spreadsheets • Creating, editing and formatting spreadsheets • Using formulas and functions in spreadsheets • Creating charts and graphs	8	5	16	0
IV	Presentations using Open Source Software	8	3	12	0

	• Introduction to Open Source Software for presentations • Creating, editing and formatting presentations • Adding multimedia and animations to presentations				
Total		28	15	60	0

Mode of In-Semester assessment (12 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

Students will complete the following tasks:

1. Word Processing Task:
 - Create a well-formatted document with text formatting, tables, images, and implement Mail Merge.
2. Spreadsheet Task:
 - Develop a spreadsheet with data entry, calculations using formulas, and visualization through a chart or graph.
3. Presentation Task:
 - Design a 5-slide presentation using an OSS presentation tool, incorporating text, images, multimedia, and animations.

Evaluation Criteria:

- Accuracy and completeness of tasks.
- Proper use of formatting, formulas, and design elements.
- Creativity and clarity in the final output.

Laboratory Work: The laboratory work will cover hands-on exercises related to the course topics. Students will be required to use Open Source Software applications for office automation and complete assignments and projects that demonstrate their understanding of the software.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Demonstrate a clear understanding of Open Source Software and its benefits in office automation.
- CLO-2. Learn to create and format documents, insert various objects, and effectively use Mail Merge in open-source word processing software.
- CLO-3. Use open-source spreadsheet software to create and format spreadsheets, apply formulas, and create charts and graphs.
- CLO-4. Learn to design and format presentations, and incorporate multimedia and animations to enhance presentation effectiveness.

CLO-5. Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁				
Conceptual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₂ , CO ₃ , CO ₄			
Procedural Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₂ , CO ₃ , CO ₄			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	2		1		3					2
CLO2	2		1					1		1
CLO3	2		1					1	2	1
CLO4	2		1					2		
Average Contribution	2		1	0	0.75			1	0.5	1

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Bain, M. A. (2006). *Learn OpenOffice. orgSpreadsheet Macro Programming: OOoBasic and Calc Automation: a Fast and Friendly Tutorial to Writing Macros and Spreadsheet Applications*. Packt Publishing Ltd.
2. Bazzell, M. (2016). *Open source intelligence techniques: resources for searching and analyzing online information*. CreateSpace Independent Publishing Platform.

Semester-II

MAJ –5-CSC-201: Computer Organization and Mathematical Foundation-II

Course Code	: MAJ –5-CSC-201
Course Title	: Computer Organization and Mathematical Foundation-II
Nature of the Course	: MAJOR
Total Credit assigned	: 5
Component of the Course	: Lecture: 5 Practical: 0 Tutorial: 1
Distribution of Marks	: End Semester: 70 Practical: 0 In Semester: 30

Course Objectives

The main objectives of this course are:

- CO1. Understand the basic structure of a computer system
- CO2. Learn the internal working of the CPU.
- CO3. Understand different memory types and their functions..
- CO4. Study I/O techniques and device communication.
- CO5. Understand basic data concepts and descriptive statistics.
- CO6. Learn fundamental probability concepts and rules.
- CO7. Understand properties of random variables.
- CO8. Study probability functions for different types of variables.
- CO9. Analyze relationships between variables.

Course Content

Section A: Computer Organization

Unit	Content	Marks	L	P	T
I	Computer Organization • CPU and memory organization • Instruction set architecture and addressing modes • Memory reference, input-output, and interrupts • Bus architecture and interconnection structures • Input-output organization	10	11	0	2
II	Central Processing Unit • CPU registers • Instruction cycle and execution • Control unit, hardwired control and microprogrammed control • RISC and CISC architectures	8	8	0	2
III	Memory Organization • Cache memory and mapping techniques • Associative memory and its organization • Virtual memory	9	6	0	2
IV	Input-Output Organization • External devices and I/O modules	8	5	0	2

	• Programme controlled I/O and interrupt-driven I/O • Direct memory access and I/O channels				
Total		35	30	0	8

Section B: Mathematical Foundation-II

Unit	Content	Marks	L	P	T
V	Introduction and Overview: • Concepts of data, data collection methods, • the distinction between populations and samples and between population parameters and sample statistics; • the use of measures of location and variation to describe and summarize data; • population moments and their sample counterparts • Measures of central tendency, dispersion	5	6	0	2
VI	Probability Theory • Basic concepts of probability theory • Conditional probability and Bayes' theorem • Discrete and continuous probability distributions	7	6	0	2
VII	Random Variables • Random variables and their probability distributions • Expectation, variance, and moments of random variables	7	7	0	1
VIII	Probability Functions • Probability mass functions • Probability density functions	8	6	0	1
IX	Correlation and Regression • Correlation analysis and correlation coefficients • Simple linear regression	8	5	0	1
Total		35	30	0	7

Mode of In-Semester assessment (30 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

Computer Organization

1. Practical Analysis of CPU and Memory Organization

- Identify and analyze CPU components, instruction execution cycles, and memory organization using a simulation tool.

2. Case Study on RISC vs. CISC Architectures

- Compare the features, advantages, and applications of RISC and CISC processors through real-world examples.

Mathematical Foundation-II

1. Probability Distribution Application

- Solve real-world problems using discrete and continuous probability distributions.

2. Correlation and Regression Project

- Analyze a dataset to calculate correlation coefficients and develop a simple linear regression model.

Evaluation Criteria:

- Accuracy and completeness of tasks.
- Clarity in explanation and analysis.
- Proper use of statistical and computational tools.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

CLO1. Explain CPU and memory organization, instruction formats, and I/O operations..

CLO2. Describe CPU registers, instruction cycles, and control unit types.

CLO3. Explain cache, associative, and virtual memory organization.

CLO4. Describe I/O methods including interrupts and DMA.

CLO5. Summarize and describe data using measures of central tendency and dispersion.

CLO6. Apply basic and conditional probability, including Bayes' theorem.

CLO7. Compute expectation, variance, and moments of random variables.

CLO8. Differentiate and use probability mass and density functions.

CLO9. Calculate correlation and perform simple linear regression.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₅ , CO ₆ , CO ₇ , CO ₈					
Conceptual Knowledge		CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅ , CO ₆ , CO ₇ , CO ₈ , CO ₉				
Procedural Knowledge			CO ₄ , CO ₅ , CO ₆ , CO ₈ , CO ₉			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3			1			1			
CLO2	3			1			1			
CLO3	3			1			1			
CLO4	2			1			1			
CLO5				1			1		3	
CLO6				1			1		3	
CLO7				1			1		3	
CLO8				1			1		3	
CLO9				1			1		3	
Average Contribution	1.2			1			1		1.6	

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Mano, M. M. (1993). *Computer system architecture*. Prentice-Hall, Inc..
2. Stallings, W. (2016). *Computer organization and architecture: designing for performance*. Pearson Education India.
3. Hamacher, V. C., Vranesic, Z. G., Zaky, S. G., Vransic, Z., & Zakay, S. (1984). *Computer organization* (pp. 367-390). New York: McGraw-Hill.
4. Walpole, R. (2000). *Probability and statistics for engineers and scientists*.
5. Schervish, M. J., & DeGroot, M. H. (2014). *Probability and statistics* (Vol. 563). London, UK:: Pearson Education.
6. Starkings, S. (2010). *A First Course in Probability and Statistics* by BLS PrakasaRao.

Semester-II

MIN-5-CSC-201: Data Structure through C++

Course Code	: MIN –5-CSC-201
Course Title	: Data Structure Using C++
Nature of the Course	: MINOR
Total Credit assigned	: 5
Component of the Course	: Lecture: 3 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 42 Practical: 40 In Semester: 18

Course Objectives

The main objectives of this course are:

- CO1. Understand basic OOP concepts, data structures, and algorithm fundamentals.
- CO2. Learn array and linked list implementations and their analyses.
- CO3. Study stack and queue data structures along with recursion principles.
- CO4. Understand tree data structures, including binary search trees and balanced trees.
- CO5. Learn common searching and sorting algorithms and hashing techniques.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to OOP and Data Structures - Fundamental concept of OOP - Introduction to Data Structures and Algorithms - Basic Terminology and Concepts - Linear and Non Linear data structure - Abstract Data Types (ADTs) - Complexity Analysis	10	10	12	0
II	Arrays and Linked Lists - Array Implementation and Analysis - Single and Multi-dimensional Arrays - Sparse Matrices (Array and Linked Representation) - Linked List Implementation and Analysis - Singly, Doubly and Circular Lists (Array and Linked representation)	8	10	12	0
III	Stacks, Queues and Recursion - Stack Implementation and Analysis - Applications of Stack and Limitations of Array representation of stack - Infix, Prefix and Postfix expressions and their conversions - Queue Implementation and Analysis, De-queue, Priority Queues	8	8	12	0

	Recursion: Developing Recursive Definition of Simple Problems and their implementation; Advantages and Limitations of Recursion; Understanding what goes behind Recursion (Internal Stack Implementation)				
IV	Trees - Introduction to Tree as a data structure - Binary Trees, Binary Search Trees (Insertion, Deletion, Recursive and Iterative Traversals on Binary Search Trees) - Balanced Trees (Various operations on AVL Trees)	8	8	12	0
V	Searching and Sorting and Hashing - Linear Search, Binary Search - Comparison of Linear and Binary Search - Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort - Comparison of Sorting Techniques - Introduction to Hashing and Hash Tables - Collision Resolution Techniques - Separate Chaining, Open Addressing	8	9	12	0
Total		42	45	60	0

Mode of In-Semester assessment (18 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

- Implementation of Data Structures
 - Implement **arrays, linked lists, and stacks** using a programming language of choice.
 - Compare **array-based and linked-list-based** representations for efficiency.
- Expression Conversion and Evaluation
 - Develop a program to convert **infix expressions to postfix** and evaluate postfix expressions using stacks.
- Tree Implementation (3 Marks)
 - Implement **Binary Search Trees (BST)** and perform insert, delete, and traversal operations (recursive and iterative).
 - Compare **balanced (AVL) trees** with unbalanced trees in terms of efficiency.
- Sorting and Hashing Techniques
 - Implement and compare sorting algorithms (**Quick Sort, Merge Sort, and Insertion Sort**).
 - Design and implement **hashing functions** with collision resolution techniques (separate chaining and open addressing).

Evaluation Criteria:

- Code correctness** and implementation accuracy.
- Efficiency** of algorithms used.
- Understanding and explanation** of concepts.
- Proper documentation** and test cases for implementation.

Lab Assignment:

1. Implement and analyse the time complexity of various operations on an array, such as insertion, deletion, and searching.
2. Implement a singly linked list and perform operations such as insertion, deletion, and searching.
3. Implement a stack using an array and perform stack operations, such as push, pop, and peek. Analyse the time complexity of these operations.
4. Implement a queue using an array and perform queue operations, such as enqueue, dequeue, and peek. Analyse the time complexity of these operations.
5. Implement the conversion of infix expressions to postfix using a stack.
6. Implement a binary search tree and perform operations such as insertion, deletion, and searching.
7. Implement a heap and perform heap operations, such as insert, delete, and build heap.
8. Implement various sorting algorithms, such as bubble sort, selection sort, insertion sort, quicksort, and mergesort, and analyse their time complexity.
9. Implement a hash table using separate chaining collision resolution and analyse its time complexity.
10. Implement dynamic hashing and analyse its performance.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Explain fundamental OOP concepts and basic data structure terminology.
- CLO2. Implement and analyze arrays and various types of linked lists.
- CLO3. Apply stack and queue operations and solve problems using recursion.
- CLO4. Perform operations on binary search trees and balanced trees.
- CLO5. Implement and compare searching, sorting, and hashing algorithms.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅					
Conceptual Knowledge		CO ₁ , CO ₄				
Procedural Knowledge			CO ₂ , CO ₃ , CO ₅			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	3	3	1		2	2	2	1
CLO2	3	1	3	3	1		2	2	2	1
CLO3	3	1	3	3	1		2	2	2	1
CLO4	3	1	3	3	1		2	2	2	1
CLO5	3	1	3	3	1		2	2	3	1
Average Contribution	3	1	3	3	1		2	2	2.2	1

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Kanetkar, Y. (2019). *Let Us C++: Dive into the nitty-gritties of C++ language and learn why programmers prefer OOPs and C++*. BPB Publications.
2. Drozdek, A. (2005). *Data Structures and algorithms in C++*. Thomson Course Technology.
3. Langsam, Y., Augenstein, M. J., & Tenenbaum, A. M. (1996). *Data Structures using C and C++*. Prentice Hall Press.
4. Anggoro, W. (2018). *C++ Data Structures and Algorithms: Learn how to write efficient code to build scalable and robust applications in C++*. Packt Publishing Ltd.
5. Malik, D. S. (2006). *C++ Programming: From problem analysis to program design*. Course Technology Cengage Learning.

Semester-II

SEC -3-CSC-201: Dynamic Web Development

Course Code	: SEC –3-CSC-201
Course Title	: Dynamic Web Development
Nature of the Course	: SEC
Total Credit assigned	: 3
Component of the Course	: Lecture: 1 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 40 In Semester: 12

Course Objectives

The main objectives of this course are:

- CO1. Understand the basics of web development, including tools and processes.
- CO2. Learn fundamental JavaScript concepts, syntax, and DOM manipulation.
- CO3. Explore AJAX, JSON, libraries, and address JavaScript security and performance.
- CO4. Understand DHTML and XML concepts to create dynamic web applications.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to Web Development • Overview of web development • The web development process • Web development tools and technologies	3	2	8	0
II	JavaScript Basics • Introduction to JavaScript • Variables, data types, and operators • Control statements and loops • Functions and events • DOM manipulation • Debugging JavaScript	10	4	16	0
III	Advanced JavaScript • AJAX and JSON • Using JavaScript libraries (e.g., jQuery) • Security and performance issues •	7	4	16	0
IV	DHTML and XML • Introduction to DHTML • Creating and manipulating HTML elements • Styling with CSS • Animations and effects • Introduction to XML • Syntax and structure	8	5	20	0

	- Document Type Definitions (DTDs) and XML Schemas - XML parsing and transformation - Combining DHTML and XML to create dynamic web applications				
Total		28	15	60	0

Mode of In-Semester assessment (12 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

1. JavaScript Basics

- Write a program using **JavaScript variables, loops, functions, and event handling**.
- Create a simple **interactive webpage using DOM manipulation**.

2. Advanced JavaScript

- Develop a **webpage that fetches and displays data using AJAX and JSON**.
- Use **jQuery to create animations and interactive elements**.

3. DHTML Implementation

- Design a **dynamic webpage using DHTML** that modifies elements dynamically using JavaScript.
- Implement **CSS animations and effects**.

4. XML and Data Handling

- Create an **XML document with DTD or XML Schema validation**.
- Implement XML parsing and transformation to **display structured data on a webpage**.

Evaluation Criteria:

- Code functionality** and correctness.
- Creativity and interactivity** of the web applications.
- Use of best practices** in JavaScript, DHTML, and XML.
- Proper documentation** and explanation of implementations.

Laboratory Work:

- Use JavaScript to create a simple calculator that performs basic arithmetic operations.
- Create a JavaScript program that validates user input in a form and provides feedback to the user.
- Use the jQuery library to create an image slideshow with navigation buttons.
- Create a DHTML webpage with a navigation bar, dropdown menus, and animations.
- Create an XML document with a DTD that defines a set of elements and attributes, and write a program that reads the XML and transforms it into an HTML webpage.
- Use AJAX to retrieve data from a server and display it dynamically on a webpage.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Describe the web development process and common tools used.
- CLO2. Write JavaScript code using variables, control statements, functions, and manipulate the DOM.

- CLO3. Implement AJAX and JSON techniques and use JavaScript libraries effectively.
 CLO4. Create dynamic web pages using DHTML, CSS styling, and XML parsing.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₂ , CO ₃ , CO ₄				
Conceptual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₂ , CO ₃ , CO ₄			
Procedural Knowledge		CO ₂	CO ₂ , CO ₃ , CO ₄			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	2	2	1	3	2	2	2	2
CLO2	3	1	2	2	1	3	2	2	2	2
CLO3	3	1	2	2	1	3	2	2	2	2
CLO4	3	1	2	2	1	3	2	2	2	2
Average Contribution	3	1	2	2	1	3	2	3	2	2

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Flanagan, D. (2011). *JavaScript: the definitive guide*. " O'Reilly Media, Inc."
2. Bibeault, B., De Rosa, A., & Katz, Y. (2015). *jQuery in Action*. Simon and Schuster.
3. Vohra, A. (2007). *Pro XML Development with Java Technology*. Apress.

Semester-II

MDC –3-CSC-201: Working with Multimedia

Course Code	: MDC –3-CSC-201
Course Title	: Working with Multimedia
Nature of the Course	: MDC
Total Credit assigned	: 3
Component of the Course	: Lecture: 1 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 40 In Semester: 12

Course Objectives

- CO1. Understand multimedia types and the role of open-source software in multimedia production.
- CO2. Learn graphic design principles and create graphics using tools like GIMP and Inkscape..
- CO3. Use open-source audio tools to record, edit, and manage audio for multimedia projects.
- CO4. Apply video production techniques using software such as Blender and OpenShot.
- CO5. Create 2D and 3D animations using open-source animation software.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to Multimedia and Open-Source Software • Definition and types of multimedia • Overview of open-source software for multimedia production • Importance of open-source software in modern society	4	3	8	0
II	Graphic Design with Open-Source Software • Introduction to graphic design principles • Designing graphics for multimedia projects with open-source software such as GIMP and Inkscape • Image optimization and preparation for web use	6	3	12	0
III	Audio Production with Open-Source Software • Introduction to audio production tools and techniques with open-source software such as Audacity and Ardour • Recording and editing audio for multimedia projects • Mixing, mastering and file management	6	3	12	0
IV	Video Production with Open-Source Software • Introduction to video production tools and	6	3	16	0

	techniques with open-source software such as Blender, OpenShot, and Shotcut • Capturing and editing video for multimedia projects • Video formats, codecs, and rendering				
V	Animation with Open-Source Software • Introduction to animation tools and techniques with open-source software such as Synfig and Blender • Creating 2D and 3D animations • Animated graphics and motion graphics	6	3	12	
Total		28	15	60	0

Mode of In-Semester assessment (12 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

Graphic Design with Open-Source Software

- Create a logo or banner design using GIMP or Inkscape.
- Optimize images for web use by adjusting resolution, compression, and file formats.

2. Audio Production with Open-Source Software

- Record and edit a short audio clip using Audacity or Ardour, applying effects like noise reduction and equalization.
- Mix and master an audio track for a multimedia project.

3. Video Production with Open-Source Software

- Capture and edit a short video clip using OpenShot or Shotcut, adding transitions and effects.
- Convert the video into different formats and codecs for web or mobile compatibility.

3. Animation with Open-Source Software

4. Create a simple 2D animation using Synfig with keyframes and motion effects.

- Develop a basic 3D animation in Blender, applying textures and simple movements.

Evaluation Criteria:

- **Creativity and originality** in multimedia projects.
- **Technical proficiency** in using open-source tools.
- **Quality and optimization** of multimedia content.
- **Proper workflow documentation** and project explanation.

Laboratory Work:

1. Explore different types of multimedia, such as images, audio, video, and animation, and their applications in various fields.
2. Design a logo or a poster for a local event or a student club. Apply design principles such as colour theory, typography, and layout to create a visually appealing and effective design.

3. Create a series of graphics for a social media campaign. Experiment with different styles and formats, such as infographics, memes, or GIFs, and optimize them for web use.
4. Record and edit a short podcast episode. Apply editing techniques such as noise reduction, equalization, and compression to enhance the audio quality and clarity.
5. Mix and master a song or a sound effect. Use effects such as reverb, delay, and chorus to create a desired sonic atmosphere, and export the file in a suitable format for distribution.
6. Create a sound design for a short film or a video game. Experiment with different sounds and layers to convey a mood or a narrative, and synchronize them with the visuals using a video editing software.
7. Plan and shoot a short film or a documentary using a smartphone or a camera. Apply editing techniques such as trimming, cutting, and crossfading to create a coherent and engaging story.
8. Create a motion graphics sequence. Use keyframe animation, particle effects, or other features to illustrate a concept or a message, and export the sequence as a video or an image sequence.
9. Create a 2D character animation. Use bone rigs, vector shapes, and tweening to create a lively and expressive animation, and export it as a video or an image sequence.
10. Create a 3D model and animation. Use modeling tools, materials, and lighting to create a realistic or stylized 3D object or environment, and animate it using keyframes or physics simulation.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Explain multimedia types and the significance of open-source software in multimedia.
- CLO2. Design and optimize graphics using open-source tools like GIMP and Inkscape.
- CLO3. Record, edit, and manage audio using software such as Audacity and Ardour.
- CLO4. Capture and edit videos using open-source video production tools.
- CLO5. Create 2D and 3D animations using Synfig and Blender.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁				
Conceptual Knowledge		CO ₂				
Procedural Knowledge			CO ₂ , CO ₃ , CO ₄ , CO ₅			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	1				2			1	1	1
CLO2	2				1			1	1	1
CLO3	2				2			1	1	1
CLO4	2				2			1	2	1
CLO5	2				1			1	1	1
Average Contribution	1.8				1.6			1.0	1.2	1

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Li, Q., & Shih, T. K. (Eds.). (2009). *Ubiquitous multimedia computing*. CRC Press.
2. Lecarme, O., & Delvare, K. (2013). *The book of GIMP: A complete guide to nearly everything*. No Starch Press.
3. Caudron, R., Nicq, P. A., & Valenza, E. (2016). *Blender 3D: Designing Objects*. Packt Publishing Ltd.
4. Parekh, R. (2006). *Principles of multimedia* (p. 727). Tata McGraw-Hill.
5. Vaughan, T. (2014). *Multimedia: Making it work*. McGraw-Hill Education Group.

Semester-III

MAJ -4-CSC-301: Operating Systems

Course Code	: MAJ –4-CSC-301
Course Title	: Operating Systems
Nature of the Course	: MAJOR
Total Credit assigned	: 4
Component of the Course	: Lecture: 3 Practical: 1 Tutorial: 0
Distribution of Marks	: End Semester: 42 Practical: 20 In Semester: 18

Course Objectives

The main objectives of this course are:

- CO-1. Understand basic functions, types, and historical development of operating systems.
- CO-2. Learn process/thread concepts, scheduling, and deadlock management.
- CO-3. Study memory allocation techniques, file systems, and device management.
- CO-4. Understand synchronization methods, critical sections, and inter-process communication.
- CO-5. Explore security policies, authentication, and fundamentals of distributed operating systems.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to Operating Systems Basic OS functions, types of operating systems, process control and real-time systems, historical evolution of operating systems	4	5	0	0
II	Process and Thread Management System view of the process and resources, process abstraction, process hierarchy, threads, threading issues, thread libraries, process scheduling, deadlocks	10	10	10	0
III	Memory Management, File and I/O Management - Physical and virtual address space, memory allocation strategies (fixed and variable partitions, paging, segmentation), virtual memory - Directory structure, file operations, file allocation methods, device management	15	16	12	0
IV	Synchronization and Concurrency Critical sections, semaphores, monitors, methods for inter-process communication, deadlocks	5	8	8	0
V	Protection and Security & Distributed Systems	8	6	0	0

	• Policy mechanism, authentication, internal access authorization, access control • Distributed file systems, distributed process management, distributed synchronization and concurrency, distributed security				
Total		42	45	30	0

Mode of In-Semester assessment (18 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

B. Sessional activity:

1. Process and Thread Management

- Implement a **multi-threaded program** to demonstrate thread creation and synchronization using **thread libraries (e.g., Pthreads in C or Java threads)**.
- Simulate a **process scheduling algorithm** (e.g., Round Robin, FCFS, or Priority Scheduling).

2. Memory Management

- Implement a **paging simulation** to demonstrate memory allocation and virtual memory management.
- Develop a program to simulate **First Fit, Best Fit, and Worst Fit** memory allocation strategies.

3. File and I/O Management

- Simulate a **file allocation method** (e.g., contiguous, linked, or indexed file allocation).
- Implement a **basic shell script** to perform file operations (create, delete, rename, and search files).

4. Synchronization and Concurrency

- Implement **producer-consumer problem** using semaphores.
- Simulate **deadlock detection and avoidance** using the Banker's Algorithm.

5. Protection, Security, and Distributed Systems

- Implement a **simple authentication mechanism** using password-based access control.
- Demonstrate **distributed process synchronization** using a message-passing mechanism.

Evaluation Criteria:

- **Correctness and efficiency** of implemented algorithms.
- **Understanding of OS concepts** in process, memory, and file management.
- **Effectiveness of synchronization mechanisms** in concurrent programming.
- **Implementation and documentation quality** of the assigned tasks.

Lab Assignments:

1. Process Management

- Implement process creation and scheduling algorithms using C/C++ language

2. Memory Management

- Implement paging and segmentation algorithms using C/C++ language

3. File and I/O Management

- Implement basic file operations and I/O system calls using C/C++ language

4. Synchronization and Concurrency

- Implement synchronization and concurrency mechanisms using semaphores and monitors using C/C++ language

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Describe the core functions and types of operating systems and their evolution.
 CLO2. Explain process/thread management, scheduling algorithms, and deadlock concepts
 CLO3. Demonstrate memory allocation methods, file operations, and device management techniques.
 CLO4. Apply synchronization mechanisms and inter-process communication methods.
 CLO5. Illustrate access control, security measures, and basic concepts of distributed systems.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁				
Conceptual Knowledge	CO ₁	CO ₂ , CO ₃	CO ₂			
Procedural Knowledge		CO ₂	CO ₃ , CO ₄			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3		1	1	1	1	2	1	1	1
CLO2	3		1	1	1	1	2	1	1	1
CLO3	3		1	1	1	2	2	1	1	1
CLO4	3		1	2	1	1	2	1	1	1
CLO5	3		1	2	1	1	2	1	2	2
Average Contribution	3		1	1.4	1	1.2	2	1	1.2	1.2

(3=High, 2= Moderate 1= Low)

Suggested readings:

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating system concepts essentials*. Wiley Publishing.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems*. Pearson Education, Inc..
- Stallings, W. (2009). *Operating systems internals and design principles*. Pearson Prentice Hall

Semester-III

MAJ –4-CSC-302: Database Management System

Course Code	: MAJ –4-CSC-302
Course Title	: Database Management System
Nature of the Course	: MAJOR
Total Credit assigned	: 4
Component of the Course	: Lecture: 3 Practical: 1 Tutorial: 0
Distribution of Marks	: End Semester: 42 Practical: 20 In Semester: 18

Course Objectives

The main objectives of this course are:

- CO1. Understand the fundamentals, types, and architecture of database systems.
- CO2. Learn to model data using ER diagrams and relational model concepts.
- CO3. Write SQL queries and understand normalization and database design principles.
- CO4. Understand transaction properties, concurrency control methods, and recovery mechanisms.
- CO5. Explore database recovery methods and implement basic database security measures.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to Databases Overview of Databases, Importance and Role of Databases, Characteristics of Databases, Evolution of Database Systems, Types of Databases, Database Management System Architecture.	8	8	0	0
II	Entity Relationship (ER) Modeling and Relational Data Model Introduction to ER Modeling, Entities, Attributes, and Relationships, Cardinality Ratio and Participation constraints, ER Diagrams, Enhanced Entity Relationship (EER) Model, ER Modeling Tools, Relational Model Concepts, Relational Model Constraints, Relational Algebra.	10	10	0	0
III	SQL and Database Design SQL Data Definition Language (DDL), SQL Data Manipulation Language (DML), SQL Queries and Sub-queries, Introduction to Database Design, Anomalies in database design, Functional Dependencies, Normalisation and Normal Forms (1NF, 2NF, 3NF), Boyce-Codd Normal Form (BCNF).	10	10	30	0
IV	Transaction Management and Concurrency Control, and Recovery Techniques Introduction to Transactions, Properties of Transactions	8	9	0	0

	(ACID), Transaction State, Concurrency Control Techniques, Locking and Timestamp-based Protocols, Introduction to Database Recovery, Recovery Techniques: Undo and Redo Operations.				
V	Unit 5: Database Recovery Techniques and Security Database Security: Authentication and Authorization.	6	8	0	0
Total		42	45	30	0

Mode of In-Semester assessment (18 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

- Entity-Relationship (ER) Modeling
 - Design an **ER diagram for a real-world application** (e.g., Library Management System, Hospital Database).
 - Use an **ER modeling tool** (e.g., MySQL Workbench, Lucidchart) to create an ER model.
- SQL Query Implementation
 - Write **SQL queries** to perform **CRUD (Create, Read, Update, Delete) operations** on a database.
 - Implement **SQL subqueries and joins** to retrieve meaningful data from multiple tables.
- Normalization and Database Design
 - Normalize a given unstructured database into **1NF, 2NF, 3NF, and BCNF**.
 - Identify **functional dependencies** and explain how normalization eliminates anomalies.
- Transaction and Concurrency Control
 - Simulate a **bank transaction system** demonstrating ACID properties.
 - Implement **lock-based and timestamp-based concurrency control mechanisms**.
- Database Security and Recovery
 - Implement **authentication and authorization** mechanisms in a database.
 - Demonstrate a **recovery technique** using an **Undo/Redo log-based approach**.

Evaluation Criteria:

- Correctness and optimization** of SQL queries and database design.
- Effectiveness of transaction control and recovery mechanisms.**
- Clarity and accuracy of ER modeling and normalization steps.**
- Security implementation techniques in database access control.**

Lab Assignments:

- Database Design:** Students will design a relational database schema for a given scenario using MySQL Workbench, identifying entities, attributes, relationships, and constraints.
- SQL Queries:** Students will write and execute SQL queries using MySQL to perform various data manipulation tasks such as insertion, deletion, updating, and querying data from the database.
- Database Implementation:** Students will implement the database schema designed in the first assignment using MySQL Server, creating tables, defining relationships, and populating the database with sample data.

4. **Basic Administration Tasks:** Students will perform basic MySQL administration tasks such as creating databases, users, and permissions, as well as taking database backups using MySQL Workbench.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Explain the importance, types, and architecture of database systems.
- CLO2. Draw ER diagrams and apply relational model concepts for data modeling.
- CLO3. Construct SQL queries and perform database normalization.
- CLO4. Demonstrate understanding of transactions, concurrency control, and recovery techniques.
- CLO5. Implement database recovery procedures and apply security mechanisms.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁					
Conceptual Knowledge		CO ₁ , CO ₂ , CO ₄	CO ₃ , CO ₅			
Procedural Knowledge			CO ₃ , CO ₄ , CO ₅			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3		1				1		2	
CLO2	3		2	2			2	1	2	
CLO3	3		2	2	1		1	1	2	
CLO4	3		2	2	2		1		2	
CLO5	3		2	2	3	1	1		2	1
Average Contribution	3		1.8	1.6	1.2	0.2	1.2	0.4	2	0.2

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Elmasri, R., & Navathe, S. B. (2004). Fundamentals of database systems: [4-th edition]..
2. Ramakrishnan, R., & Gehrke, J. (2003). Database management systems third edition..
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2011). Database system concepts.

Semester-III

MIN -2-CSC-301: PHP Programming

Course Code	: MIN -2-CSC-301
Course Title	: PHP Programming
Nature of the Course	: MINOR
Total Credit assigned	: 2
Component of the Course	: Lecture: 0 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 0 Practical: 40 In Semester: 0

Course Objectives

The main objectives of this course are:

- CO1. Understand the fundamentals of PHP, including syntax, variables, and data types.
- CO2. Learn to apply control statements and create functions in PHP programs.
- CO3. Understand how to connect PHP with MySQL and perform database CRUD operations.
- CO4. Learn techniques for processing web forms and managing user sessions in PHP.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to PHP - Overview of PHP language - Setting up development environment - PHP syntax and basic constructs - Variables, data types, and operators	0	0	10	0
II	Control Structures and Functions - Conditional statements: if, else, elseif - Looping structures: for, while, do-while - Functions: declaration, parameters, return values	0	0	12	0
III	Database Interaction with MySQL - Introduction to MySQL database - Connecting to MySQL database using PHP - Performing CRUD operations (Create, Read, Update, Delete) with PHP and MySQL - Querying data from MySQL database and displaying results	0	0	24	0
IV	Form Handling and Session Management - Handling HTML forms using PHP - Processing form data with GET and POST methods - Session management: starting, using, and destroying sessions	0	0	14	0
Total		0	0	60	0

Course Activity:

1. Setting Up PHP Environment and Basics
 - Install and configure **XAMPP or WAMP** to set up a PHP development environment.
 - Write basic PHP programs to demonstrate **variables, data types, and operators**.
2. Implementing Control Structures and Functions
 - Develop a PHP program using **if-else, loops (for, while, do-while)** for decision-making and iteration.
 - Create and call **functions** with parameters and return values.
3. Database Connectivity and CRUD Operations
 - Establish a **connection between PHP and MySQL**.
 - Perform **Create, Read, Update, and Delete (CRUD) operations** on a MySQL database using PHP.
4. Form Handling in PHP
 - Create an **HTML form** and process user inputs using **GET and POST methods**.
 - Validate form inputs and display submitted data dynamically.
5. Session Management in PHP
 - Implement **session creation, maintenance, and destruction** in a simple login/logout system.
 - Store and retrieve session data to track user activity.

Evaluation Criteria:

- **Correct implementation** of PHP syntax and control structures.
- **Effective database interactions and query handling** using MySQL.
- **Secure and structured form handling** with GET and POST methods.
- **Proper session management techniques** to maintain user authentication.

Expected Course Learning Outcomes:

By the end of the course, students will be able to

- CLO1. Explain PHP basics and demonstrate use of variables, data types, and operators.
- CLO2. Write PHP scripts using conditional statements, loops, and user-defined functions.
- CLO3. Develop PHP applications that interact with MySQL databases for data manipulation.
- CLO4. Handle HTML form submissions and implement session management in PHP.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁	CO ₂			
Conceptual Knowledge	CO ₁ , CO ₂	CO ₂	CO ₃			
Procedural Knowledge	CO ₂ , CO ₃	CO ₃ , CO ₄	CO ₃ , CO ₄			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	2	2	1	3	2	2	2	1
CLO2	3	1	2	2	1	3	2	2	2	1
CLO3	3	1	2	2	1	3	2	2	2	1
CLO4	3	1	2	2	1	3	2	2	2	1
Average Contribution	3	1	2	2	1	3	2	3	2	1

(3=High, 2= Moderate 1= Low)

Suggested readings:

7. Holzner, S. (2007). *PHP: the complete reference*. McGraw-Hill, Inc..
8. Boronczyk, T., & Psinas, M. E. (2008). *PHP and MySQL Create-Modify-Reuse*. John Wiley & Sons.

Semester-III

MIN –2-CSC-302: Probability and Statistics

Course Code	: MIN –2-CSC-302
Course Title	: Probability and Statistics
Nature of the Course	: MINOR
Total Credit assigned	: 2
Component of the Course	: Lecture: 2 Practical: 0 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 0 In Semester: 12

Course Objectives

- CO1. Understand fundamental concepts of probability and various probability distributions.
- CO2. Learn different sampling methods and the principles of sample size determination.
- CO3. Understand the concept of sampling distributions and apply the Central Limit Theorem.
- CO4. Comprehend the basics of hypothesis testing including errors and significance levels.

Course Content

Unit	Content	Marks	L	P	T
I	Overview Overview of probability, probability distribution, types of probability distribution, binomial distribution, poisson distribution, normal distribution.	9	8	0	0
II	Introduction to sampling Concept of sample and population, sampling methods, types of sampling: simple random sampling, systematic sampling, stratified sampling, cluster sampling; sample size determination, advantages and disadvantages of sampling,	6	7	0	0
III	sampling distribution: Introduction to sampling distribution, sample distribution of sample mean, sampling distribution of sample proportion, Central limit theorem, application of sampling distribution in estimation.	7	8	0	0
IV	Introduction to hypotheses testing: Null and Alternate hypotheses, Types of errors in hypotheses testing, type I and type II errors, significance level and p value.	6	7	0	0
Total		28	30	0	0

Mode of In-Semester assessment (12 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

1. Probability Distributions
 - Solve problems on **binomial, Poisson, and normal distributions**.
 - Interpret real-world applications of probability distributions.
2. Sampling Methods
 - Conduct a practical **sampling exercise** using simple random, stratified, and cluster sampling.
 - Evaluate the **advantages and disadvantages** of different sampling techniques.
3. Sampling Distribution and Estimation
 - Demonstrate the **Central Limit Theorem** through hands-on simulations.
 - Estimate population parameters from sample distributions.
4. Hypothesis Testing
 - Formulate **null and alternative hypotheses** for real-life case studies.
 - Perform hypothesis tests using **p-values and significance levels**.
 - Identify and differentiate **Type I and Type II errors** in decision-making.

Evaluation Criteria:

- **Accuracy in solving probability problems** using appropriate distributions.
- **Correct application of sampling methods** and justification for choices.
- **Clear understanding of hypothesis testing** with practical implementation.
- **Demonstration of sampling distribution concepts** with real-world examples.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Explain and identify different types of probability distributions such as binomial, Poisson, and normal.
- CLO2. Differentiate between sampling techniques and apply appropriate methods for selecting samples.
- CLO3. Demonstrate how sampling distributions work and use the Central Limit Theorem in estimation problems.
- CLO4. Formulate hypotheses and interpret the implications of Type I and Type II errors in hypothesis testing.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁				
Conceptual Knowledge	CO ₁ , CO ₂ , CO ₃	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₃ , CO ₄			
Procedural Knowledge		CO ₂ , CO ₄	CO ₂ , CO ₃ , CO ₄			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	2		1				1		3	
CLO2	2		1				1		2	
CLO3	2		1				1		3	
CLO4	2		1		2		1		3	
Average Contribution	2		1		0.5		1		2.75	

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Gupta, S. C., & Kapoor, V. K. (2020). *Fundamentals of mathematical statistics*. Sultan Chand & Sons.
2. Devore, J. L., & Carlton, M. A. (2016). *Probability and Statistics for Engineering and the Sciences: Student Solutions Manual*. Cengage Learning.
3. Freund, J. E. (1962). *Mathematical Statistics* Prentice Hall. Inc. Engle wood Cliffs, NJ.

Semester-III

SEC –3-CSC-301: Advanced Web Design

Course Code	: SEC –3-CSC-301
Course Title	: Advanced Web Design
Nature of the Course	: SEC
Total Credit assigned	: 3
Component of the Course	: Lecture: 1 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 40 In Semester: 12

Course Objectives:

- CO1. Understand the components and interactions within the web stack including clients, servers, and databases.
- CO2. Learn advanced HTML, CSS, and template techniques for dynamic web content.
- CO3. Explore website deployment strategies including CI/CD pipelines.
- CO4. Develop skills to build CRUD and RESTful APIs for web applications.
- CO5. Understand how to integrate and use external third-party APIs in web projects.

Course Content

Unit	Content	Marks	L	P	T
I	The Web Stack Overview of clients, web servers, and databases, Understanding the interaction between components in the web stack	7	3	12	0
II	Advanced HTML, CSS, and Templates Advanced HTML techniques, CSS pre-processors, Template engines for dynamic content generation	4	2	12	0
III	Website Deployment Deployment strategies: manual deployment vs. automated deployment, Continuous integration and continuous deployment (CI/CD) pipelines	5	4	12	0
IV	Building CRUD/RESTful APIs Introduction to APIs and RESTful architecture, Implementing CRUD operations with RESTful APIs	6	3	12	0
V	Working with External APIs Overview of external APIs and their usage, Integrating third-party APIs into web applications	6	3	12	0
Total		28	15	60	0

Mode of In-Semester assessment (12 Marks):

- A. Sessional Activity
- B. Sessional Tests: (two Sessional Tests)

Sessional activity:

1. Understanding the Web Stack
 - **Diagram the architecture** of a web stack (client, server, database).
 - **Analyze request-response cycles** in web applications.
2. Advanced HTML, CSS, and Templates
 - **Create responsive web pages** using CSS preprocessors (e.g., SASS).
 - **Use template engines** (e.g., Handlebars, EJS) to generate dynamic content.
3. Website Deployment
 - **Deploy a simple website** using manual methods and CI/CD pipelines.
 - **Compare different deployment strategies** and their benefits.
4. Building CRUD/RESTful APIs
 - **Develop a RESTful API** to handle CRUD operations using a backend framework (e.g., Express.js, Flask).
 - **Test API endpoints** using Postman or cURL.
5. Working with External APIs
 - **Integrate a third-party API** (e.g., weather, payment, or maps) into a web application.
 - **Handle API authentication and data fetching** using AJAX or Fetch API.

Evaluation Criteria:

- **Successful implementation of RESTful APIs** with proper CRUD functionality.
- **Ability to deploy a website** using a structured approach.
- **Effective use of HTML, CSS, and template engines** to create dynamic and interactive pages.
- **Integration of external APIs** in a functional and meaningful way.

Lab Work:

1. Web Stack Understanding:
 - Task: Study web stack components (clients, servers, databases), and their interaction.
 - Objective: Gain a thorough grasp of the web stack's role in web development.
2. Advanced HTML/CSS Practice:
 - Task: Practice advanced HTML techniques, CSS pre-processors, and dynamic template implementation.
 - Objective: Enhance proficiency in dynamic web content creation using HTML/CSS.
3. Website Deployment Exploration:
 - Task: Explore manual and automated deployment methods, including CI/CD pipeline setup.
 - Objective: Understand and implement efficient website deployment strategies.
4. CRUD/RESTful API Development:
 - Task: Learn about APIs and RESTful architecture, and implement CRUD operations.
 - Objective: Develop skills in designing and implementing RESTful APIs for web applications.
5. External API Integration:
 - Task: Explore various external APIs and integrate them into web applications.
 - Objective: Understand external API usage and gain practical experience in integration for enhanced web app functionality.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Describe the roles of clients, web servers, and databases and how they interact in the web stack.
- CLO2. Apply advanced HTML and CSS techniques and use template engines to create dynamic web pages.
- CLO3. Demonstrate knowledge of deploying websites manually and through automated CI/CD pipelines.
- CLO4. Implement RESTful APIs with CRUD operations in web applications.
- CLO5. Integrate external APIs into web applications for enhanced functionality.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁	CO ₂ , CO ₄			
Conceptual Knowledge	CO ₁	CO ₂ , CO ₃ , CO ₅	CO ₃ , CO ₅			
Procedural Knowledge		CO ₄	CO ₂ , CO ₃ , CO ₄ , CO ₅			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	2	2	1	3	2	2	2	2
CLO2	3	1	2	2	1	3	2	2	2	2
CLO3	3	1	2	2	1	3	2	2	2	2
CLO4	3	1	2	2	1	3	2	2	2	2
CLO5	3	1	2	2	1	3	2	3	2	2
Average Contribution	3	1	2	2	1	3	2	2	2	2

(3=High, 2= Moderate 1= Low)

Suggested readings:

- Schmitt, C. (2012). *Designing Web and Mobile Graphics: Fundamental concepts for web and interactive projects*. Pearson Education.
- Design, J. R. L. W. (2012). *A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics*.
- Richardson, L., Amundsen, M., & Ruby, S. (2013). *RESTful web APIs: services for a changing world*. " O'Reilly Media, Inc."

Semester-III

MDC –3-CSC-301: Introduction to Financial Accounting Package

Course Code	: MDC –3-CSC-301
Course Title	: Introduction to Financial Accounting Package
Nature of the Course	: MDC
Total Credit assigned	: 3
Component of the Course	: Lecture: 1 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 40 In Semester: 12

Course Objectives

- CO1. Understand the fundamental principles and concepts of financial accounting and the accounting cycle.
- CO2. Gain proficiency in navigating and configuring accounting software for financial operations.
- CO3. Learn how to accurately record various financial transactions in accounting systems.
- CO4. Develop skills to generate and customize financial reports for analysis and decision-making.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction to Financial Accounting - Overview of financial accounting principles and concepts - Understanding the accounting cycle: recording transactions, posting to ledgers, trial balance - Introduction to financial statements: balance sheet, income statement, cash flow statement	12	4	0	0
II	Navigating Accounting Software - Creating company profiles and configuring settings - Navigating software interface: menus, options, and functions	6	4	20	0
III	Recording Transactions - Entering and editing transactions: sales, purchases, expenses, and payments - Posting transactions to appropriate accounts and ledgers	5	4	20	0
IV	Financial Reporting - Generating financial reports: balance sheet, income statement, cash flow statement - Customizing reports and analysis in accounting software	5	3	20	0
Total		28	15	60	0

Mode of In-Semester assessment (12 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

1. Introduction to Financial Accounting
 - **Discuss financial accounting concepts** and their role in decision-making.
 - **Practice recording transactions** and preparing trial balances.
 - **Analyze real-world financial statements** from companies.
2. Navigating Accounting Software
 - **Create a company profile** and configure initial settings in accounting software.
 - **Explore menus, features, and functionalities** of accounting software through guided tutorials.
3. Recording Transactions
 - **Enter transactions** for sales, purchases, expenses, and payments.
 - **Post transactions** to appropriate accounts and ledgers.
 - **Verify and correct errors** in transaction entries.
4. Financial Reporting
 - **Generate key financial reports** (balance sheet, income statement, cash flow statement).
 - **Customize and analyze reports** for business insights.
 - **Compare manual and automated financial reporting** using accounting software.

Evaluation Criteria:

- **Accuracy in recording transactions** and maintaining financial records.
- **Proficiency in using accounting software** for bookkeeping and reporting.
- **Ability to interpret financial statements** for decision-making.

Lab Work:

1. **Transaction Recording Practice:** Students will utilize accounting software to record sample transactions, including sales, purchases, expenses, and payments. They will then post these transactions to appropriate accounts and ledgers, reinforcing understanding of the accounting cycle.
2. **Software Navigation and Setup:** Students will familiarize themselves with the accounting software interface by navigating menus, options, and functions. They will also create company profiles and configure settings to understand software customization.
3. **Financial Reporting:** Through hands-on exercises, students will generate financial reports such as balance sheets, income statements, and cash flow statements using the accounting software. They will customize reports and perform basic analysis.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Explain key financial accounting principles and complete the accounting cycle including preparation of trial balances.
- CLO2. Demonstrate effective use of accounting software to set up company profiles and navigate its features.
- CLO3. Accurately enter and post sales, purchases, expenses, and payments to relevant accounts.
- CLO4. Generate and interpret financial statements such as balance sheets, income statements, and cash flow reports using accounting software.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁	CO ₃			
Conceptual Knowledge		CO ₁ , CO ₂	CO ₄			
Procedural Knowledge		CO ₂	CO ₂ , CO ₃ , CO ₄			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	2		1	1	2			1	1	1
CLO2	2		1	1	1			2	1	1
CLO3	2		1	1	2			1	1	1
CLO4	2		1	1	2			2	2	2
Average Contribution	2		1	1	1.75			1.5	1.25	1.25

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Grewal, T. S., & Gupta, S. C. (2003). *Introduction to Accountancy*. S. Chand Publishing.
2. Asok, K. N. Mastering Tally ERP 9: Basic Accounts, Invoice, Inventory.

Semester-IV

MAJ –4-CSC-401: Discrete Mathematics

Course Code	: MAJ –4-CSC-401
Course Title	: Discrete Mathematics
Nature of the Course	: MAJOR
Total Credit assigned	: 4
Component of the Course	: Lecture: 4 Practical: 0 Tutorial: 0
Distribution of Marks	: End Semester: 56 Practical: 0 In Semester: 24

Course Objectives

The main objectives of this course are:

- CO1. Understand foundational mathematical structures including sets, relations, functions, and principles of counting.
- CO2. Familiarize with basic algebraic structures such as groups, rings, and fields.
- CO3. Introduce the core concepts of graph theory and their applications in computing.
- CO4. Understand and apply the principles of propositional logic and logical reasoning.

Course Content

Unit	Content	Marks	L	P	T
I	Introduction: Sets - finite and Infinite sets, uncountably Infinite Sets; functions, relations, Properties of Binary Relations, Closure, Partial Ordering Relations; counting - Pigeonhole Principle, Permutation and Combination; Mathematical Induction, Principle of Inclusion and Exclusion.	12	15	0	0
II	Algebraic Structure: - Operations, semi groups, Groups, Sub-groups, Normal Sub-groups & Homomorphism: simple concepts - Rings, Integral Domains & Fields –concepts only	10	15	0	0
III	Graph Theory: Basic Terminology, Models and Types, multi-graphs and weighted graphs, Graph Representation, Graph Isomorphism, Connectivity, Euler and Hamiltonian Paths and Circuits, Trees, Basic Terminology and properties of Trees, Introduction to Spanning Trees, Planar Graphs, Graph Coloring	24	18	0	0
IV	Propositional Logic Logical Connectives, Well-formed Formulas, Tautologies, Equivalences, Inference Theory	10	12	0	0
Total		56	60	0	0

Mode of In-Semester assessment (24 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

1. Introduction to Sets and Counting Principles
 - **Solve problems on set operations** and understand countability.
 - **Apply the pigeonhole principle** to real-world scenarios.
 - **Practice permutation and combination problems** for different cases.
2. Algebraic Structures
 - **Identify and differentiate groups, rings, and fields** through examples.
 - **Perform basic operations on semigroups and groups** with hands-on problems.
 - **Explore homomorphisms** and their properties through guided exercises.
3. Graph Theory
 - **Draw and analyze different types of graphs**, including multigraphs and weighted graphs.
 - **Apply connectivity concepts** to real-world networks.
 - **Solve Eulerian and Hamiltonian circuit problems** with examples.
 - **Work on spanning trees and graph coloring problems** to understand their significance.
4. Propositional Logic
 - **Construct and simplify logical expressions** using logical connectives.
 - **Use inference theory to prove logical equivalences** and tautologies.
 - **Solve problems using well-formed formulas** to develop logical reasoning skills.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Apply set theory, relations, functions, and combinatorics to solve discrete mathematical problems.
- CLO2. Illustrate the structure and properties of algebraic systems including groups and rings.
- CLO3. Analyze and interpret graphs and trees, determine connectivity, and apply graph traversal algorithms.
- CLO4. Construct and evaluate logical statements using propositional logic, tautologies, and inference rules.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₂				
Conceptual Knowledge		CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₃ , CO ₄			
Procedural Knowledge			CO ₄			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3		3	1			2	2	2	
CLO2	3		1	1			2	1	2	
CLO3	3		3	2		2	2	2	3	
CLO4	3		3	1			2	1	2	
Average Contribution	3		2.5	1.25		.5	2	1.5	2.5	

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Rosen, K. H., Trembly, J. P., & Manohar, R. (2015). "Discrete Mathematics and its Applications", Special Indian edition, Tata McGraw-Hill Pub. Co. Ltd., New Delhi, (2007).
2. Lipschutz, S., & Lipson, M. L. (2007). Schaum's outline of discrete mathematics. (*No Title*).
3. Rosen, K. H. (2007). Discrete mathematics and its applications sixth edition.
4. Albertson, M. O., & Hutchinson, J. P. (1988). *Discrete mathematics with algorithms*. John Wiley & Sons, Inc..

Semester-IV

MAJ -4-CSC-402: OOP with JAVA

Course Code	: MAJ –4-CSC-402
Course Title	: OOP with JAVA
Nature of the Course	: MAJOR
Total Credit assigned	: 4
Component of the Course	: Lecture: 3 Practical: 1 Tutorial: 0
Distribution of Marks	: End Semester: 42 Practical: 20 In Semester: 18

Course Objectives

The main objectives of this course are:

- CO1. Introduce the fundamental features and control structures of Java programming.
- CO2. Enable understanding of arrays, strings, and wrapper classes in Java.
- CO3. Develop object-oriented programming skills using classes, objects, constructors, and methods.
- CO4. Explain the principles of inheritance and interface implementation in Java.
- CO5. Familiarize students with packages, exception handling, multithreading, and I/O programming in Java.

Course Content

Unit	Content	Marks	L	P	T
I	Basics of Java: - Features of Java, Byte Code, and Java Virtual Machine - JDK, JRE, Data types, and Operators - Control Statements: If-else, switch, while, do-while, for, for-each, break, continue	6	6	4	0
II	Array and String: - Single and Multidimensional Array - String class, StringBuffer class - Operations on strings, Command line argument - Use of Wrapper Class	6	6	6	0
III	Classes, Objects, and Methods: - Class, Object, Object reference, Constructor, Constructor Overloading - Method Overloading, Recursion - Passing and Returning object form Method, new operator, this and static keyword - finalize() method, Access control, modifiers - Nested class, Inner class, Anonymous inner class, Abstract class	9	10	6	0
IV	Inheritance and Interfaces: Use of Inheritance, Inheriting Data members and	6	12	6	0

	Methods, constructor in inheritance • Multilevel Inheritance, method overriding, super keyword • Stop Inheritance - Final keywords • Creation and Implementation of an interface, Interface reference • Introduction to Interfaces, implementation of interfaces • Comparison between Abstract Class and interface				
V	Package, Exception Handling, Multithreaded Programming, and IO Programming • Use of Package, Import statement, Built-in & user-defined packages • Exception and Error handling • Use of try, catch, throw, throws, and finally blocks • Built-in Exception, Custom exception, Throwable Class • Overview of Multithread programming • Introduction to Stream, Byte Stream, Character stream, Readers and Writers • File Class, File InputStream, File OutputStream • InputStreamReader, OutputStreamWriter, FileReader, FileWriter • Buffered Reader	15	11	8	0
Total		42	45	30	0

Mode of In-Semester assessment (18 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

1. Basics of Java
 - Write and execute **simple Java programs** using JDK and JRE.
 - Implement **control statements (if-else, switch, loops)** with real-world examples.
 - Demonstrate the use of **operators** through coding exercises.
2. Array and String
 - Create and manipulate **single and multidimensional arrays** in Java.
 - Perform **string operations** using String and StringBuffer classes.
 - Write programs to demonstrate **command-line arguments** and **wrapper classes**.
3. Classes, Objects, and Methods
 - Develop **Java classes with constructors and methods**.
 - Implement **method overloading and recursion** with examples.
 - Demonstrate **object creation, object passing, and returning** in methods.
 - Work with **access control modifiers**, static and this keywords.
4. Inheritance and Interfaces
 - Implement **single and multilevel inheritance** in Java.
 - Demonstrate **method overriding, use of super, and final keyword**.
 - Write Java programs using **interfaces and abstract classes**.
 - Compare **abstract classes and interfaces** through practical examples.
5. Package, Exception Handling, Multithreading, and I/O Programming

- Develop **user-defined packages** and use built-in Java packages.
- Implement **exception handling mechanisms** using try, catch, throw, throws, and finally.
- Write programs to handle **custom exceptions and built-in exceptions**.
- Demonstrate **multithreading concepts** with thread creation and synchronization.
- Work with **Java I/O classes**, including FileReader, FileWriter, BufferedReader, and Streams.

Evaluation Criteria:

- Conceptual understanding of Java fundamentals and OOP principles.
- Ability to develop and debug Java applications involving arrays, strings, and classes.
- Skill in exception handling, multithreading, and file operations.
- Implementation of object-oriented design principles in Java programs.

LAB Work:

1. Write a program to convert rupees to dollars. (1 Rupee = 1/60 Dollar)
2. Write a program to calculate the percentage marks of a student if marks of 6 subjects are given.
3. Write a program to enter two numbers and perform mathematical operations (addition, subtraction, multiplication, division) on them.
4. Write a program to find the length of a string and print its second half.
5. Write a program to accept a line and check how many consonants and vowels are there in the line.
6. Write a program to count the number of words that start with capital letters.
7. Write a program to find if a given number or string is a palindrome or not.
8. Create a class that asks the user to enter a sentence and displays the count of each vowel type in the sentence. The program should continue until the user enters the word "quit". Display the total count of each vowel for all sentences.
9. Write an interactive program to print a string entered in a pyramid form.
10. Write an interactive program to print a diamond shape based on user input.
11. Create a class called Student. Write a student manager program to manipulate student information from files using FileInputStream and FileOutputStream.
12. Refine the student manager program to manipulate student information from files using BufferedReader and BufferedWriter.
13. Refine the student manager program to manipulate student information from files using DataInputStream and DataOutputStream.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Apply Java syntax and control structures to write basic Java programs.
- CLO2. Manipulate arrays and string objects efficiently using built-in classes and methods.
- CLO3. Implement object-oriented programming principles like encapsulation, method overloading, and inner classes.
- CLO4. Demonstrate inheritance, method overriding, and the use of interfaces and abstract classes.

CLO5. Build Java programs with proper exception handling, multithreading, and file I/O operations.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅	CO ₁ , CO ₂				
Conceptual Knowledge		CO ₂ , CO ₃ , CO ₄	CO ₃ , CO ₄ , CO ₅			
Procedural Knowledge		CO ₅	CO ₁ , CO ₃ , CO ₄ , CO ₅			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	3	3		1	3	1	1	2
CLO2	3	1	3	3		1	3	1	2	2
CLO3	3	1	3	3		1	3	1	3	2
CLO4	3	1	3	3		1	3	1	2	2
CLO5	3	1	3	3		1	3	1	3	2
Average Contribution	3	1	3	3		1	3	1	2.2	2

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Schildt, H., & Skrien, D. (2012). *Java Programming: A Comprehensive Introduction*. McGraw-Hill Higher Education.
2. Balaguruswamy, E. (2014). *Programming with Java-A Primer*. McGraw-Hill Professionals.
3. Horstmann, C. S. (2024). *Core java, volume I: fundamentals*. Pearson Education.

Semester-IV

MAJ –4-CSC-403: Theory of Computing

Course Code	: MAJ –4-CSC-403
Course Title	: Theory of Computing
Nature of the Course	: MAJOR
Total Credit assigned	: 4
Component of the Course	: Lecture: 3 Practical: 0 Tutorial: 1
Distribution of Marks	: End Semester: 56 Practical: 0 In Semester: 24

Course Objectives

The main objectives of this course are:

- CO1. To understand the concepts and applications of finite automata including DFA, NFA, and their equivalence.
- CO2. To familiarize with formal language theory, grammars, and the Chomsky hierarchy.
- CO3. To develop skills to use regular expressions and grammars for language representation and processing.
- CO4. To understand context-free grammars (CFGs) and Pushdown Automata (PDA), and their interrelations.
- CO5. To explore the Turing Machine model of computation and understand the fundamental limits of computability.

Course Content

Unit	Content	Marks	L	P	T
I	Finite Automata - Deterministic Finite Automata (DFA): Formal definition, simpler notations (state transition diagram, transition table), language of a DFA - Nondeterministic Finite Automata (NFA): Definition of NFA, language of an NFA - Equivalence between DFA and NFA - Finite Automata with Epsilon Transitions, Eliminating Epsilon transitions - Minimization of finite automata - Finite automata with output (Moore and Mealy machines) and Interconversion.	12	9	0	3
II	Formal Languages and Grammars - Basics of formal languages - The Chomsky hierarchy - Relations between different classes of languages	10	10	0	2

III	Regular Languages - Regular Expressions (RE): Introduction, Identities of Regular Expressions, Finite Automata and Regular Expressions- Converting from DFA's to Regular Expressions, Converting Regular Expressions to Automata. - Regular Grammars: Definition, regular grammars and FA, FA for regular grammar, Regular grammar for FA; Proving languages to be non-regular -Pumping lemma.	12	9	0	3
IV	Context-Free Grammars and Pushdown Automata - Derivation Trees, Sentential Forms, Rightmost and Leftmost derivations of Strings. - Ambiguity in CFG's. - Simplification of CFG's. - Chomsky Normal Form, Greibach Normal Form. - Introduction to pushdown automata. - Deterministic Pushdown Automata (DPDA), Nondeterministic Pushdown Automata (NPDA). - Equivalence between DPDA and NPDA. - Acceptance of CFL, Acceptance by Final State and Acceptance by Empty stack and its Equivalence. - Equivalence of CFG and PDA.	12	10	0	4
V	Turing Machines - Definition and formal description of Turing machines. - Turing machine variants: multitape, nondeterministic, and alternating Turing machines. - Universal Turing machines. - Halting problem and undecidability.	10	7	0	3
Total		56	45	0	15

Mode of In-Semester assessment (24 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

1. Finite Automata

- **Construct DFA and NFA** for given languages.
- Convert **NFA to DFA** and **eliminate epsilon transitions** in finite automata.
- **Minimize finite automata** using state reduction techniques.
- Implement **Moore and Mealy machines** and **interconvert them**.

2. Formal Languages and Grammars

- Classify **languages based on Chomsky hierarchy**.
- Identify relationships between **regular, context-free, context-sensitive, and recursively enumerable languages**.

3. Regular Languages

- Convert **regular expressions to finite automata and vice versa**.
- Develop **regular grammars and construct corresponding finite automata**.

- Use the **pumping lemma to prove non-regularity** of languages.
4. Context-Free Grammars and Pushdown Automata
- Generate **sentential forms and derivation trees** for given CFGs.
 - Convert CFGs into **Chomsky Normal Form (CNF) and Greibach Normal Form (GNF)**.
 - **Design and implement PDA** for recognizing context-free languages.
 - Demonstrate **acceptance by final state and empty stack** in PDA.
 - Establish the **equivalence of CFGs and PDAs**.
5. Turing Machines
- **Construct Turing machines** for simple problems.
 - Explore **multitape and nondeterministic Turing machines** through simulations.
 - Understand **the concept of universal Turing machines**.
 - Analyze the **halting problem and undecidability** through real-world examples.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Design and analyze finite automata (DFA, NFA, ϵ -NFA) and convert between them.
- CLO2. Classify languages using the Chomsky hierarchy and explain the relationships among different types of grammars.
- CLO3. Construct regular expressions and regular grammars for formal languages, and prove non-regularity using the pumping lemma.
- CLO4. Construct context-free grammars and design pushdown automata for context-free languages, including transformations between them.
- CLO5. Explain the structure and behavior of Turing machines and demonstrate understanding of decidability and the halting problem.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅	CO ₁ , CO ₂ , CO ₄ , CO ₅				
Conceptual Knowledge		CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅	CO ₃ , CO ₄ , CO ₅			
Procedural Knowledge		CO ₅	CO ₁ , CO ₃ , CO ₄ , CO ₅			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	2	1			2		1	
CLO2	3	1	2	1			2		1	
CLO3	3	1	2	1			2		1	
CLO4	3	1	2	1			2		1	
CLO5	3	1	2	1			2		1	
Average Contribution	3	1	2	1			2		1	

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Lewis, H. R., & Papadimitriou, C. H. (1998). Elements of the Theory of Computation. *ACM SIGACT News*, 29(3), 62-78.
2. Mishra, K. L. P., & Chandrasekaran, N. (2006). *Theory of computer science: automata, languages and computation*. PHI Learning Pvt.Ltd..
3. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2001). Introduction to automata theory, languages, and computation. *AcmSigact News*, 32(1), 60-65.
4. Linz, P., & Rodger, S. H. (2022). *An introduction to formal languages and automata*. Jones & Bartlett Learning.

Semester-IV

DSE –4-CSC-401: Introduction to Management and Financial Accounting

Course Code	: DSE –4-CSC-401
Course Title	: Introduction to Management and Financial Accounting
Nature of the Course	: DSE
Total Credit assigned	: 4
Component of the Course	: Lecture: 3 Practical: 1 Tutorial: 0
Distribution of Marks	: End Semester: 42 Practical: 20 In Semester: 18

Course Objectives

The main objectives of this course are:

- CO1. To provide a foundational understanding of accounting principles, concepts, and processes including journalizing and ledger posting.
- CO2. To enable the preparation and analysis of financial statements and understand depreciation accounting.
- CO3. To introduce the fundamentals of financial management, including capital structure and working capital.
- CO4. To develop skills in budgeting, capital budgeting, and break-even analysis for effective financial planning.

Course Content

Unit	Content	Marks	L	P	T
I	Accounting Fundamentals - Exploring the meaning and definition of accounting, identifying parties interested in accounting, and discussing branches of accounting. - Understanding accounting concepts, conventions, and basic terminologies, including classification of accounts. - Learning journal entry, ledger posting, and balancing of ledger, along with the significance of subsidiary books and preparation of cash book (Triple Column).	10	9	16	0
II	Financial Statements and Depreciation - Clarifying the concept and objectives of trial balance and the preparation of financial statements such as Trading and Profit and Loss Accounts, and Balance Sheet for sole trading concerns. - Discussing the classification of assets and liabilities and accounting for depreciation, including its meaning, causes, and treatment. - Introducing accounting software like Tally with basic concepts.	12	12	14	0
III	Financial Management and Working Capital Defining financial management and its objectives, and	10	12	0	0

	examining the functions of financial management. Exploring the concept of capital structure, computation of Cost of Capital, and managing working capital, including the operating cycle and sources of working capital.				
IV	Budgeting and Capital Budgeting - Discussing budget and budgetary control, including their objectives, classification, advantages, and characteristics. - Preparing production/sales and cash budgets, and understanding the concept, importance, and methods of capital budgeting. - Introducing Marginal costing, Cost-Volume-Profit analysis, and Break-even Point concepts.	10	12	0	0
Total		42	45	30	0

Mode of In-Semester assessment (18 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

- Accounting Fundamentals
 - Analyze different branches of accounting** and their significance.
 - Classify accounts based on **accounting principles and conventions**.
 - Practice **journal entry, ledger posting, and balancing ledgers**.
 - Prepare **cash books (Triple Column)** and understand subsidiary books.
- Financial Statements and Depreciation
 - Prepare trial balance and financial statements** for sole proprietors.
 - Classify **assets and liabilities** in a balance sheet.
 - Calculate and apply depreciation** using different methods.
 - Hands-on session with Tally** for recording transactions.
- Financial Management and Working Capital
 - Explore **capital structure concepts** and compute the **cost of capital**.
 - Analyze **working capital management**, including the operating cycle.
 - Study different **sources of working capital** and their impact on financial decisions.
- Budgeting and Capital Budgeting
 - Prepare production/sales and cash budgets**.
 - Discuss **budgetary control mechanisms** and their advantages.
 - Perform **capital budgeting techniques** such as NPV, IRR, and Payback Period.
 - Apply **cost-volume-profit analysis** to decision-making.

Lab Work:

- Transaction Recording Practice:** Students will utilize accounting software to record sample transactions, including sales, purchases, expenses, and payments. They will then post these transactions to appropriate accounts and ledgers, reinforcing understanding of the accounting cycle.
- Software Navigation and Setup:** Students will familiarize themselves with the accounting software interface by navigating menus, options, and functions. They will also create company profiles and configure settings to understand software customization.

3. **Financial Reporting Workshop:** Through hands-on exercises, students will generate financial reports such as balance sheets, income statements, and cash flow statements using the accounting software. They will customize reports and perform basic analysis.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO1. Explain the meaning, purpose, and scope of accounting and record basic accounting transactions using journal entries and ledgers.
- CLO2. Prepare trial balances, trading and profit & loss accounts, and balance sheets, and apply accounting treatment for depreciation.
- CLO3. Analyze financial management functions, determine cost of capital, and evaluate working capital requirements.
- CLO4. Prepare different types of budgets and apply capital budgeting techniques and marginal costing concepts in decision-making.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂ , CO ₃	CO ₁ , CO ₂				
Conceptual Knowledge		CO ₂ , CO ₃	CO ₂ , CO ₃ , CO ₄			
Procedural Knowledge		CO ₄	CO ₁ , CO ₂ , CO ₄			
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	2		1		2			1	1	1
CLO2	2		1		2			1	1	1
CLO3	2		1		2			1	2	1
CLO4	2		1		2			2	2	1
Average Contribution	2		1		2			1.25	1.5	1

(3=High, 2= Moderate 1= Low)

Suggested readings:

1. Edmonds, T., & Tsay, B. Y. (2014). *Fundamental Managerial Accounting Concepts 7e*. McGraw Hill.
2. Banerjee, B. (2012). *Financial policy and management accounting*. PHI Learning Pvt. Ltd..
3. Bargate, K. (2012). The readability of managerial accounting and financial management textbooks. *Meditari Accountancy Research*, 20(1), 4-20.

Semester V

MAJ-5-CSC-501: Computer Networks

Course Code	: MAJ-5-CSC-501
Course Title	: Computer Networks
Nature of the Course	: MAJOR
Total Credit assigned	: 5
Component of the Course	: Lecture: 3 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 42 Practical: 40 In Semester: 18

Course Objectives:

The main objectives of this course are:

- CO1. To introduce the basic concepts, architecture, models, topologies, transmission modes, and networking devices in computer networks.
- CO2. To provide foundational understanding of data link layer operations, Ethernet standards, addressing schemes, and network protocols including ARP and VLANs.
- CO3. To explore IP routing, transport protocols, congestion control mechanisms, QoS, and fundamental network security practices.
- CO4. To examine application layer protocols, real-world network applications, emerging networking technologies, and network management tools.

Course Content:

Unit	Content	Marks	L	P	T
I	Introduction to Computer Networks • Overview of Computer Networks: Definition and Objectives, Applications and Examples Network Components and Architecture • Network Models: OSI Model: Layers and Functions, TCP /IP Model: Layers and Functions Comparison between OSI and TCP/IP Models • Network Topologies: Physical vs. Logical Topologies, Common Topologies: Star, Ring, Bus, Mesh, Hybrid, Advantages and Disadvantages of Each Topology • Data Transmission: Analog vs. Digital Signals, Transmission Modes: Simplex, Half-Duplex, Full-Duplex, Bandwidth and Latency • Networking Devices: Routers, Switches, Hubs, Bridges, Gateways, Functions and Configurations of Each Device.	12	13	16	0
II	Data Link Layer and Networking Protocols • Data Link Layer Fundamentals: Functions of the Data Link Layer, Framing, Error Detection, and Error Correction, Flow Control Mechanisms. • Ethernet: Ethernet Standards and Frame Structure, MAC	10	11	14	0

	Addressing and ARP, Ethernet Switching: Basic Concepts and Methods • Network Protocols: Introduction to TCP/IP Protocol Suite, IP Addressing: IPv4 and IPv6 Subnetting and CIDR Notation • Address Resolution Protocol (ARP): ARP Operation and Table, ARP Spoofing and Security Considerations • Virtual LANs (VLANs): Concept of VLANs, VLAN Tagging and Configuration, Benefits and Use Cases				
III	Network Layer and Transport Layer • Network Layer: IP Routing: Static vs. Dynamic Routing, Routing Protocols: RIP, OSPF, BGP, Network Address Translation (NAT) • Transport Layer: TCP vs. UDP: Characteristics and Use Cases, TCP Handshake and Connection Management, Flow Control and Congestion Control in TCP • Congestion Control Algorithms: Techniques: Slow Start, Congestion Avoidance, Fast Retransmit, Fast Recovery, TCP Variants: TCP Reno, TCP Vegas. • Quality of Service (QoS): QoS Principles and Mechanisms, Differentiated Services (DiffServ) and Integrated Services (IntServ) • Network Security Fundamentals: Threats and Vulnerabilities, Basic Security Mechanisms: Firewalls, VPNs, Encryption	10	14	16	0
IV	Application Layer and Emerging Technologies • Application Layer Protocols: HTTP/HTTPS: Structure and Operation, FTP, SMTP, POP3, IMAP: Protocols and Uses, DNS: Domain Name System and Resolution • Network Applications: Web Browsing, Email Communication, File Transfer, Voice over IP (VoIP) and Streaming. • Emerging Technologies: Software-Defined-Networking (SDN), Network Function Virtualization (NFV), Internet of Things (IoT) and Its Impact on Networking • Network Management: SNMP: Simple Network Management Protocol, Network Monitoring Tools and Techniques. • Future Trends in Networking: 5G and Beyond, Network Automation and Artificial Intelligence in Networking	10	16	14	0
Total		42	45	60	0

Mode of In-Semester assessment (18 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional Activities:

1. **Network Design Exercise:** Design a network using various topologies (e.g., star, mesh) and select appropriate devices (router, switch, hub).
2. **Protocol Configuration Task:** Implement and configure basic protocols like TCP/IP, ARP, and VLANs on a simulated network.
3. **Troubleshooting Exercise:** Troubleshoot a given network setup, identifying issues with IP addressing, routing, and device configuration.
4. **Packet Sniffing Lab:** Use network monitoring tools (e.g., Wireshark) to capture and analyze network traffic for different protocols (HTTP, FTP, etc.).
5. **Routing Protocol Simulation:** Simulate routing protocols (RIP, OSPF) and demonstrate dynamic IP routing in a small network.
6. **Network Security Assessment:** Set up basic security measures like firewalls and VPNs, and assess network vulnerabilities.

Suggested list of LAB Assignments (not limited to):

1. Configure Basic Network Settings:
 - a) IP Address Configuration
 - b) Subnet Mask and Gateway Settings
2. Implement Network Protocols:
 - a) Write a simple Python script to perform DNS resolution.
 - b) Implement a basic HTTP client-server application.
3. Network Simulation:
 - a) Use network simulation tools (e.g., Cisco Packet Tracer) to design and simulate network topologies.
 - b) Configure routers and switches in a simulated environment.
4. Performance Measurement:
 - a) Measure network performance using tools like `ping`, `traceroute`, and `iperf`.
 - b) Analyze network traffic using Wireshark.
5. Implement VLANs:
 - a) Configure VLANs on a switch and verify using simulation tools.
6. Set Up a Simple Web Server:
 - a) Deploy a basic web server and configure HTTP/HTTPS access.
7. Network Security Lab:
 - a) Implement basic firewall rules and VPN configurations.
 - b) Perform vulnerability scanning and analyze results.
8. Network Troubleshooting:
 - a) Diagnose and resolve common network issues.
 - b) Use troubleshooting commands and techniques to fix connectivity problems

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Explain the structure and functioning of computer networks, compare OSI and TCP/IP models, identify network topologies, and describe the roles of common networking devices.
- CLO-2. Analyze data link layer functions, implement subnetting, understand MAC addressing, and configure basic VLANs and ARP settings.

- CLO-3. Differentiate between routing protocols and transport layer protocols, explain congestion control strategies, and apply basic network security concepts such as firewalls and VPNs.
- CLO-4. Describe the operation of application layer protocols, evaluate the role of technologies like SDN, IoT, and 5G, and apply tools for monitoring and managing network performance.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₂				
Conceptual Knowledge		CO ₂ , CO ₃ , CO ₄	CO ₃ , CO ₄	CO ₃ , CO ₄		
Procedural Knowledge		CO ₃ , CO ₄	CO ₂ , CO ₃ , CO ₄	CO ₃ , CO ₄		
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3		1	1		1	2	2	2	1
CLO2	3		1	1		1	2	2	2	1
CLO3	3		2	1	3	1	2	3	3	2
CLO4	3		2	1	2	2	3	2	3	2
Average Contribution	3		1.5	1	1.25	1.25	2.25	2.25	2.5	1.5

(3=High, 2= Moderate 1= Low)

Text Books:

1. Tanenbaum, A. S. Computer Networks 5th ed.(2011)..
2. Kurose, J. F., & Ross, K. W. (2007). Computer networking: A top-down approach edition. Addison Wesley, 12..

Reference Books:

1. Forouzan, B. A. (2007). Data communications and networking. Huga Media..
2. Peterson, L. L., & Davie, B. S. (2007). Computer networks: a systems approach. Elsevier.

Semester V

Course Code	: MAJ-4-CSC-502
Course Title	: Artificial Intelligence
Nature of the Course	: MAJOR
Total Credit assigned	: 4
Component of the Course	: Lecture: 3 Practical: 1 Tutorial: 0
Distribution of Marks	: End Semester: 42 Practical: 20 In Semester: 18

Course Objectives:

- CO1. Understand the fundamentals of AI, including intelligent agents and problem-solving techniques.
- CO2. Learn advanced search algorithms and optimization techniques like A*, Minimax, and genetic algorithms.
- CO3. Gain knowledge of logical reasoning, uncertainty handling, and AI-based planning systems.
- CO4. Explore domains of AI such as machine learning, computer vision, robotics, and expert systems, with an understanding of legal and ethical issues.

Course Content:

Unit	Content	Marks	L	P	T
I	Introduction to AI What is AI? Intelligent Agents: Agents and environment, the concept of Rationality, the nature of environment, the structure of Agents. Knowledge-Based Agents: Introduction to Knowledge-Based Agents, The Wumpus World as an Example World. Problem-solving: Problem-solving agents	10	11	6	0
II	Advanced Search Techniques Uninformed Search: DFS, BFS, Iterative Deepening Search. Informed Search: Best First Search, A* search, AO* search. Adversarial Search & Games: Two-player zero-sum games, Minimax Search, Alpha-Beta pruning. Constraints and Constraint Satisfaction Problems (CSPs), Backtracking search for CSP. Evolutionary Search Techniques: Introduction to evolutionary algorithms, Genetic algorithms, Applications of evolutionary search in AI.	11	12	9	0
III	Logical Reasoning and Uncertainty Logic: Propositional logic, First-order predicate logic, Propositional versus first-order inference, Unification and lifting. Inference: Forward chaining, Backward chaining, Resolution, Truth maintenance systems. Introduction to Planning: Blocks World problem, Strips; Handling Uncertainties: Non-monotonic reasoning, Probabilistic reasoning, Introduction to Fuzzy set theory	11	11	8	0

IV	Domains and Applications of AI Domains in AI: Introduction to Machine Learning, Computer Vision, Robotics, Natural Language Processing, Deep Neural Networks, and their Applications. Expert Systems: The architecture and role of expert systems include two case studies. Legal and Ethical Issues: Concerns related to AI	10	11	7	0
		42	45	30	0

Mode of In-Semester assessment (18 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional Activities:

- Search Algorithm Implementation:** Implement DFS, BFS, and A* search algorithms and solve simple path finding problems.
- Minimax Algorithm with Alpha-Beta Pruning:** Simulate a two-player game (e.g., Tic-Tac-Toe) using Minimax search and optimize it with Alpha-Beta pruning.
- CSP Problem Solving:** Solve a Constraint Satisfaction Problem (CSP) using backtracking search (e.g., Sudoku or map coloring).
- Genetic Algorithm Application:** Implement a basic genetic algorithm to solve an optimization problem (e.g., finding the best-fit solution to a traveling salesman problem).
- Logical Reasoning Exercise:** Use forward and backward chaining to infer facts in a knowledge-based agent (e.g., BlocksWorld problem).
- Probabilistic Reasoning Simulation:** Implement a simple probabilistic reasoning task (e.g., Bayesian network for disease diagnosis).

Suggested list of LAB Assignments (not limited to):

Prerequisites: Basic understanding of algorithms and data structures (e.g., trees, graphs, lists). Proficiency in Python programming, including libraries like NLTK for NLP tasks.

LAB Experiments

The lab experiments may be implemented in Python. Libraries like NLTK, Tensorflow and Keras may be used for Machine learning experiments.

Suggested list of Experiments (not limited to):

- Demonstrate basic problem-solving using Breadth-First Search on a simple grid.
- Implement Depth-First Search (DFS) on a small graph.
- Solve the Water Jug Problem using Breadth First Search (BFS).
- Implement a Hill Climbing search to find the peak in a numeric dataset.
- Apply the A* Search algorithm to find the shortest path in a 4x4 grid.
- Implement the Minimax search algorithm for 2-player games. You may

- use a game tree with 3 plies.
7. Solve the 4–Queens Problem as a CSP backtracking problem.
 8. Use constraint propagation to solve a Magic Square puzzle.
 9. Apply optimization techniques to find the maximum value in a list.
 10. Represent and evaluate propositional logic expressions.
 11. Implement a basic rule-based expert system for weather classification.
 12. Implement a basic AI agent with simple decision-making rules.
 13. Implement a basic Rule-Based Chatbot.
 14. Using Python NLTK, perform the following Natural Language Processing (NLP) tasks for text content.
 - a) Tokenizing
 - b) Filtering Stop Words
 - c) Stemming
 - d) Part of Speech tagging
 - e) Chunking
 - f) Named Entity Recognition(NER)
 15. Perform Image classification for a given dataset using CNN. You may use Tensorflow /Keras.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Identify and explain the concepts of AI, intelligent agents, and their environments.
- CLO-2. Implement and evaluate various search algorithms like DFS, BFS, A*, and genetic algorithms.
- CLO-3. Apply logical reasoning techniques and handle uncertainties using probabilistic and fuzzy set theory.
- CLO-4. Understand and apply AI techniques in real-world domains such as NLP, robotics, and expert systems.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₄					
Conceptual Knowledge		CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₂ , CO ₃	CO ₂ , CO ₃ , CO ₄		
Procedural Knowledge			CO ₂	CO ₂		
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3		2	2	1		2		3	
CLO2	3		3	3			2	2	3	
CLO3	3		3	3	1		2	2	3	
CLO4	2		3	3	2		2	3	3	2
Average Contribution	2.75		2.75	2.75	1		2	1.75	3	0.5

(3=High, 2= Moderate 1= Low)

Text Books:

1. Nilsson, N. J. (1998). *Artificial intelligence: a new synthesis*. Elsevier.
2. Patterson, D. (1990). *Introduction to artificial intelligence and expert systems*. Prentice-Hall, Inc.
3. Russell, S., & Norvig, P. (2010). *Artificial intelligence: a modern approach*. 3rd. Upper Saddle River, EUA: Prentice-Hall.

Semester V

MAJ-4-CSC-503: Software Engineering

Course Code	: MAJ-4-CSC-503
Course Title	: Software Engineering
Nature of the Course	: MAJOR
Total Credit assigned	: 4
Component of the Course	: Lecture: 3 Practical: 0 Tutorial: 1
Distribution of Marks	: End Semester: 56 Practical: 00 In Semester: 24

Course Objectives:

- CO1. To understand the evolving role of software, different process models, and agile methodologies for software development.
- CO2. To learn software requirements engineering, including functional and non-functional requirements, and the process of elicitation and validation.
- CO3. To explore software design concepts, testing strategies, and quality management practices for creating high-quality software products.
- CO4. To understand project planning, risk management, release management, and product sustenance in the software lifecycle.

.Prerequisites: Basic understand of Software, Applications, Programming fundamentals

Course Content:

Unit	Content	Marks	L	P	T
I	- The evolving role of software, changing nature of software, layered technology, a process framework, Process models: The waterfall model, incremental process models, evolutionary process models, spiral model. Agile software development: Agility Principles, Agile methods, Plan-driven and agile development, Extreme programming, Scrum, A Tool Set for the Agile Process.	13	10	0	4
II	Software Requirements Engineering: Functional and non-functional requirements, the software requirements document, Requirements specification, Requirements engineering processes, Requirements elicitation and analysis, Requirements validation, Requirements management. Risk management: Reactive Vs proactive risk strategies, software risks, risk identification, risk projection, risk refinement, RMMM, RMMM plan. Project planning- Software pricing, Plan-driven development, Project scheduling, Agile planning, Estimation techniques.	15	11	0	4
III	Design: Design process and design quality, design concepts, the design model, software architecture, data design, architectural design, Basic structural modeling, class	15	13	0	4

	diagrams, sequence diagrams, collaboration diagrams, use case diagrams, component diagrams. • Testing Strategies: A strategic approach to software testing, test strategies for conventional software, black-box and white-box testing, validation testing, system testing, the art of debugging. • Product metrics: Software quality, metrics for analysis model, metrics for design model, metrics for source code, metrics for testing, metrics for maintenance.				
IV	• Quality Management: Quality concepts, software quality assurance, software reviews, formal Technical reviews, statistical software quality assurance, software reliability. • Release Management: Release planning, development and build plans, release strategies, risk management, and post-deployment monitoring. • Product sustenance: Maintenance, updates, End of life, migration strategies.	13	11	0	3
Total		56	45	0	15

Mode of In-Semester assessment (24 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

- **Process Model Comparison:** Compare and analyze different software development models (Waterfall, Agile, Incremental)
- **Requirements Elicitation:** Conduct a mock session to gather and validate software requirements.
- **Risk Management Exercise:** Identify risks and create a risk management plan (RMMM) for a software project.
- **Design Diagram Creation:** Create class, sequence, and use case diagrams for a software design.
- **Testing Strategy Development:** Develop test cases using black-box and white-box testing strategies.
- **Quality Assurance Review:** Perform a technical review focusing on software quality metrics.
- **Release Planning Simulation:** Simulate the planning and monitoring of a software release.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Apply various software development models, including agile and traditional models, to real-world projects.
- CLO-2. Demonstrate the ability to collect, analyze, and validate software requirements, ensuring the development of quality software.

CLO-3. Design software architectures, create appropriate diagrams, and apply testing strategies for quality assurance.

CLO-4. Effectively manage project risks, plan releases, and maintain software products through their lifecycle.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂	CO ₁ , CO ₂				
Conceptual Knowledge		CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₂ , CO ₃ , CO ₄	CO ₃ , CO ₄		
Procedural Knowledge		CO ₂ , CO ₃ , CO ₄	CO ₂ , CO ₃ , CO ₄	CO ₃ , CO ₄		
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	3	3	1	1	2	2	1	1
CLO2	3	1	3	2	2	1	2	2	2	1
CLO3	3	2	3	2	1	1	2	3	2	1
CLO4	2	1	3	3	2	1	2	2	1	1
Average Contribution	2.75	1.25	3	2.5	1.5	1	2	2.25	1.5	1

(3=High, 2= Moderate 1= Low)

Text Books

1. Software Engineering, N.S.Gill, Khanna Publishing House, 2023(AICTE Recommended Textbook)
2. Software Engineering, Ian Somerville, 9th edition, Pearson education.
3. Software Engineering A practitioner's Approach, 8th edition, Roger S Pressman, Bruce R.Maxim. McGraw Hill Education, 2015.

Reference Books

1. Stephen Schach, Software Engineering 7th ed, McGraw-Hill, 2007
2. Software Engineering: Principles and Practice HansvanVliet

SEMESTER –V

Course Code	: DSE-3-CSC-501
Course Title	: Programming with Python
Nature of the Course	: DSE
Total Credit assigned	: 3
Component of the Course	: Lecture: 2 Practical: 1 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 20 In Semester: 12

Course Objectives:

- CO1. Understand the fundamentals of Python programming including syntax, data types, strings, and control flow statements..
- CO2. Develop skills to define and use functions and manipulate core data structures such as lists, tuples, and dictionaries.
- CO3. Apply file handling, exception management, and data visualization techniques using Python libraries.

Course Content:

Unit	Content	Marks	L	P	T
I	Introduction: History and Application areas of Python; Structure of Python Program; Identifiers and Keywords; Operators and Precedence; Basic Data Types and type conversion; Statements and expressions; Input/Output statements. Strings: Creating and Storing Strings, Built-in functions for strings; string operators, String slicing and joining; Formatting Strings. Control Flow Statements: Conditional Flow statements; Loop Control Statements; Nested control Flow; continue and break statements, continue, Pass and exit.	8	8	11	0
II	Functions: Built-In Functions, Function Definition and call; Scope and Lifetime of Variables, Default Parameters, Command Line Arguments; Lambda Functions; Assert statement; Importing User defined module; Mutable and Immutable objects: Lists, Tuples and Dictionaries; Commonly used Functions on Lists, Tuples and Dictionaries. Passing Lists, tuples and Dictionaries as arguments to functions. Using Math and Numpy module for list of integers and arrays.	10	12	9	0
III	Files: Types of Files; Creating, Reading and writing on Text and Binary Files; The Pickle Module, Reading and Writing CSV Files. Reading and writing of csv and JSON files. Exception Handling: Try-except-else-finally block, raise statement, hierarchy of exceptions, adding exceptions. Data	10	10	10	0

	visualization: Plotting various 2D and 3D graphics; Histogram; Pi charts; Sine and cosine curves.				
Total		28	30	30	0

Mode of In-Semester assessment (12 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional activity:

- **String Manipulation Exercise:** Practice string creation, slicing, joining, and formatting using built-in string functions.
- **Control Flow Project:** Create a program that uses conditional and loop control statements, including nested flows, `continue`, and `break`.
- **Function Implementation:** Write functions with different parameter types (default, command-line, lambda), and use scope and lifetime concepts.
- **Data Structures Challenge:** Implement operations on lists, tuples, and dictionaries, passing them as arguments to functions.
- **File Handling Task:** Read from and write to text, binary, CSV, and JSON files, using the `Pickle` module for serialization.
- **Exception Handling Simulation:** Build a program with custom exceptions, using `try-except` blocks and the `raise` statement.
- **Data Visualization Project:** Create visualizations (histograms, pie charts, sine/cosine curves) using Python libraries like Matplotlib

Suggested list of LAB Assignments (not limited to):

1. Write a program to find whether a number is a prime number.
2. Write a program to print m raise to power n, where m and n are read from the user.
3. Write a program having a parameterised function that returns True or False depending on whether the parameter passed is even or odd.
4. Write a program to print the summation of the following series upto n terms: $1-2+3-4+5-6+7 \dots n$
5. Write a menu driven program to perform the following operations on strings using string built in functions.
 - Find the frequency of a character in a string.
 - Replace a character by another character in a string.
 - Remove the first occurrence of a character from a string.
 - Remove all occurrences of a character from a string.
6. Write a program that accepts two strings and returns the indices of all the occurrences of the second string in the first string as a list. If the second string is not present in the first string, then it should return -1
7. Using Numpy module write menu driven program to do following
 - Create an array filled with 1's.
 - Find maximum and minimum values from an array
 - Dot product of 2 arrays.
 - Reshape a 1-D array to 2-D array.

8. Write a function that takes a sentence as input from the user and calculates the frequency of each letter. Use a variable of dictionary type to maintain the count.
9. Consider a tuple `t1=(1,2,5,7,9,2,4,6,8,10)`. Write a program to perform following operations:
 - Print contents of `t1` in 2 separate lines such that half values come on one line and other half in the next line.
 - Print all even values of `t1` as another tuple `2`.
 - Concatenate a tuple `2= (11,13,15)` with `t1`.
 - Return maximum and minimum value from `t1`.
10. Write a function that reads a file `file1` and copies only alternative lines to another file `file2`. Alternative lines copied should be the odd numbered lines.
11. Write a Python program to handle a Zero Division Error exception when dividing a number by zero.
12. Write a program that reads a list of integers from the user and throws an exception if any numbers are duplicates.
13. Write a program that makes use of a function to display sine, cosine, polynomial and exponential curves.
14. Take as input in the months and profits made by a company ABC over a year. Represent this data using a line plot. Generated line plot must include X axis label name = Month Number and Y axis label name = Total profit.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Demonstrate proficiency in Python syntax, data types, control flow, and functions.
- CLO-2. Apply concepts of mutable and immutable objects, including lists, tuples, and dictionaries, and use libraries like Numpy for data manipulation.
- CLO-3. Manage files, handle exceptions, and create basic data visualizations using Python.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁				
Conceptual Knowledge		CO ₁ , CO ₂	CO ₂			
Procedural Knowledge			CO ₂ , CO ₃	CO ₃		
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	3	3	1	1	2	1	1	1
CLO2	3	1	3	3	1	1	2	1	2	1
CLO3	3	1	3	3	1	1	2	1	3	1
Average Contribution	3	1	3	3	1	1	2	1	2	1

(3=High, 2= Moderate 1= Low)

Text Books:

1. Venkatesh, NagarajuY, Introduction to Python Programming, Khanna Publishing House, 2021.
2. Jeeva Jose, Introduction to Computing & Problem Solving With PYTHON, Khanna Publishing House, 2023.
3. SheetalTaneja& Naveen kumar: Python Programming a Modular approach– A Modular approach with Graphics, Database, Mobile and Web applications, Pearson, 2017.

Reference Books:

- Think Python, by Allen Downey, 2nd edition, 2015, O'Reilly.
https://drive.google.com/file/d/1p9Pul6d5UvnQrO9-Q-LE2_p4YvMk5cIg/view
- An introduction to Python for absolute beginners, by Bob Dowling, Cambridge Univ.
- Introduction to Computation and Programming using Python, by John Guttag, 2nd edition, 2016, PHI India.

SEMESTER –VI

Course Code	: MAJ-5-CSC-601
Course Title	: Project
Nature of the Course	: MAJOR
Total Credit assigned	: 5
Component of the Course	: Lecture: 0 Practical: 5 Tutorial: 0
Distribution of Marks	: End Semester: 0 Practical: 70 In Semester: 30

Course Objectives:

- CO1. To enable students to apply the theoretical and practical knowledge acquired throughout the BCA program in developing a comprehensive software solution, simulating real-world problems from industry, academia, or research. The project aims to foster skills in system analysis, design, development, testing, documentation, and deployment through adherence to standard software development practices.

Course Content:

The Bachelor of Computer Applications (BCA) program aims to prepare students for roles such as programmers, system designers, and software engineers in the IT industry. The curriculum provides comprehensive theoretical and practical knowledge in core computer science areas. A six-month major project in the final semester allows students to apply this knowledge to develop quality software solutions.

Steps to be followed to design and implement the project :

- Apply theoretical and practical techniques to solve real-life problems in industry, academia, or research.
- Understand and describe the System Development Life Cycle (SDLC) phases related to their project.
- Evaluate system requirements and problem definitions.
- Collect and analyse information for requirements gathering.
- Conduct feasibility studies including cost-benefit, technical, time, and operational feasibility.
- Use fact-finding methods for data collection.
- Construct and evaluate Data Flow Diagrams (DFDs), data dictionaries, decision trees, and decision tables.
- Create and analyse system flowcharts, Entity-Relationship (ER) diagrams, and state transition diagrams.
- Prepare Software Requirement Specifications (SRS) and hardware requirements.
- Plan and design functional and non-functional system requirements.
- Design system inputs, outputs, validity checks, and user interfaces.
- Perform coding and documentation of the project.
- Apply testing techniques and strategies.
- Generate project reports.
- Deploy the project in real or lab environments.
- Identify maintenance procedures and plan future enhancements.

- Include appendices as necessary in the project report.

Types of Projects

- Real-life projects in industry, research labs, educational institutions, or software companies are preferred.
- Alternatively, students may propose and develop a project with supervisor approval.
- Projects may be carried out on-campus or at approved organizations.

Mode of In-Semester assessment (30 Marks):

1. Project Proposal Submission
 - Students submit a synopsis including:
 - Project title
 - Objectives
 - Proposed tools and technologies
 - Preliminary analysis diagrams (DFD, ER, etc.)
 - Reviewed and approved by internal supervisor.
2. Phase I Presentation – Project Planning & Analysis
 - Activities:
 - Problem definition and scope
 - Feasibility study (technical, operational, financial)
 - Data collection methods
 - Preliminary system analysis (DFDs, ER diagrams, flowcharts)
 - Evaluation based on clarity, feasibility, and planning.
3. Phase II Presentation – System Design & Development
 - Activities:
 - SRS preparation
 - Database design and data structures
 - UI/UX design
 - Process logic of each module
 - Coding of main modules
 - Evaluation based on design completeness and implementation progress.
4. Phase III Presentation – Testing & Deployment
 - Activities:
 - Implementation of functional modules
 - Testing techniques and error logs
 - Validation checks
 - Report generation
 - Deployment (on lab or real environment)
 - Evaluation includes testing results, project functionality, and readiness.
5. Documentation Submission (Draft)
 - Submission of partial documentation including:
 - Introduction
 - Analysis
 - Design
 - Coding samples
 - Testing strategies
6. Final Presentation & Viva

- Complete project demonstration in front of evaluation panel.
- Viva based on:
 - Project concept and problem understanding
 - Design and coding process
 - Tools and technologies used
 - Challenges faced and solutions
 - Future enhancements

7. Final Report Submission

- Two hard copies of the bound report.
- Email containing:
 - Source code
 - Executables
 - Database
 - Screenshots
 - Documentation (Word + PDF format)

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

CLO-1. By the end of the BCA Project, students will be able to design, develop, test, and document a real-world software application by applying theoretical and practical knowledge of computer science, demonstrating proficiency in system analysis, software development life cycle (SDLC), user interface design, coding, testing strategies, and deployment, while adhering to professional standards of documentation, teamwork, and project presentation.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge						
Conceptual Knowledge						
Procedural Knowledge			CO ₁	CO ₁	CO ₁	CO ₁
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	3	3	3	3	2	3	3	2	3
Average Contribution	3	3	3	3	3	2	3	3	2	3

(3=High, 2= Moderate 1= Low)

SEMESTER –VI

Course Code	: MAJ-4-CSC-602
Course Title	: Computer Graphics
Nature of the Course	: MAJOR
Total Credit assigned	: 4
Component of the Course	: Lecture: 3 Practical: 1 Tutorial: 0
Distribution of Marks	: End Semester: 42 Practical: 20 In Semester: 18

Course Objectives:

- CO1. To introduce the foundational concepts of computer graphics systems, including basic elements, applications, and hardware components.
- CO2. To develop the ability to implement fundamental algorithms for drawing and clipping 2D graphical primitives.
- CO3. To enable students to apply 2D and 3D geometric transformations and viewing techniques in computer graphics.
- CO4. To provide knowledge of geometric modeling techniques for representing curves and surfaces.
- CO5. To introduce methods for visible surface determination, rendering techniques, and basic animation principles.

Course Content:

Unit	Content	Marks	L	P	T
I	Introduction to Computer Graphics • Overview of Computer Graphics Systems • Basic Elements of Computer Graphics • Applications of Computer Graphics • Architecture of Display Devices: ◦ Raster Scan Display ◦ Random Scan Display • Input and Output Devices in Graphics Systems	9	9	0	0
II	Drawing and Clipping Primitives • Raster Scan Algorithms for Drawing Primitives: ◦ Line Drawing ◦ Circle Drawing ◦ Ellipse Drawing • Polygon Filling Techniques • Clipping Algorithms: ◦ Line Clipping ◦ Polygon Clipping	9	10	10	0
III	Transformations and Viewing • 2D Geometric Transformations • 3D Geometric Transformations • Viewing Transformations: ◦ 2D Viewing	10	10	10	0

	○ 3D Viewing ○ Projection Methods: ▪ Parallel Projection ▪ Perspective Projection ○ Vanishing Points				
IV	Geometric Modeling • Polygon Mesh Representations • Cubic Polynomial Curves: ○ Hermite Curves ○ Bézier Curves	7	8	10	0
V	Visible Surface Determination and Surface Rendering • Visible Surface Algorithms: ○ Z-buffer Algorithm ○ List-Priority Algorithm ○ Area Subdivision Algorithm • Illumination and Shading Models • RGB Color Model • Basics of Computer Animation	7	8	0	0
Total		42	45	30	0

Mode of In-Semester assessment (18 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional Activities:

- **Lab Exercises**
 - Implement line, circle, and ellipse drawing algorithms.
 - Apply clipping and polygon filling techniques.
 - Perform 2D/3D transformations and projections using OpenGL or similar tools.
- **Mini Project**
 - Develop a small graphics-based application (e.g., animation, simple game, drawing tool).
- **Assignments**
 - Write and analyze code for curves, projections, and shading models.
- **Quiz/Viva**
 - Assess theoretical and practical understanding through periodic quizzes and vivas.
- **Presentation/Case Study**
 - Explore and present real-world applications or graphics tools (e.g., Blender, Unity).

Suggested list of LAB Assignments (not limited to):

1. Introduction to graphics environment setup (e.g., OpenGL, C/C++, or Python with libraries).
2. Drawing basic shapes: points, lines, rectangles, and circles.
3. Implement DDA (Digital Differential Analyzer) algorithm.
4. Implement Bresenham's Line Drawing algorithm.

5. Implement Bresenham's Circle Drawing algorithm.
6. Implement Midpoint Ellipse Drawing algorithm.
7. Implement Scan-line Polygon Fill algorithm.
8. Implement Boundary Fill and Flood Fill algorithms.
9. Implement Cohen-Sutherland Line Clipping algorithm.
10. Implement Sutherland-Hodgman Polygon Clipping algorithm.
11. Apply Translation, Rotation, Scaling, Reflection, and Shearing on 2D objects.
12. Use transformation matrices and composite transformations.
13. Apply basic 3D transformations (Translation, Rotation, Scaling).
14. Represent and manipulate 3D objects using transformation matrices.
15. Implement 2D and 3D viewing transformations.
16. Demonstrate Parallel and Perspective Projections with sample 3D objects.
17. Generate Hermite and Bézier Curves using control points.
18. Visualize and manipulate curves.
19. Implement Z-buffer algorithm for hidden surface removal.
20. Demonstrate List-Priority or Area Subdivision algorithms (conceptually or via code).
21. Demonstrate Phong or Gouraud Shading models (can be visual or code-based).
22. Apply RGB color models in rendering.
23. Create a simple animation (e.g., bouncing ball, moving object).
24. Use frame-by-frame rendering and transformations for motion.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1.** Explain the architecture and components of computer graphics systems, including display and I/O devices.
- CLO-2.** Implement raster algorithms and clipping techniques for 2D graphics primitives.
- CLO-3.** Apply geometric and viewing transformations to 2D and 3D graphical objects, including projection techniques.
- CLO-4.** Model 3D shapes using polygon meshes and represent curves using Hermite and Bézier methods.
- CLO-5.** Apply algorithms for visible surface determination, describe shading and illumination models, and explain basic animation concepts.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁	CO ₁				
Conceptual Knowledge		CO ₁ , CO ₄ , CO ₅	CO ₃ , CO ₄ , CO ₅	CO ₄ , CO ₅		
Procedural Knowledge		CO ₂ , CO ₃ , CO ₅	CO ₂ , CO ₃ , CO ₄ , CO ₅	CO ₃ , CO ₅		
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3		2	2		1	2	2	1	1
CLO2	3		2	2		1	2	2	2	1
CLO3	3		2	2		1	2	2	2	1
CLO4	3		2	2		1	2	2	2	1
CLO5	3		2	2		1	2	3	3	1
Average Contribution	3		2	2		1	2	2.1	2	1

(3=High, 2= Moderate 1= Low)

Text Books:

1. Foley, J. D., Van Dam, A., Feiner, S. K., & Hughes, J. F. (2003). *Computer graphics: Principles and practice* (2nd ed., in C). Pearson Education.
2. Hearn, D., & Baker, M. P. (2004). *Computer graphics with OpenGL* (C version, 2nd ed.). Pearson Education.
3. Rogers, D. F., & Adams, J. A. (1990). *Mathematical elements for computer graphics* (2nd ed.). McGraw-Hill.
4. Hill, F. S. Jr. (2003). *Computer graphics using OpenGL*. Pearson Education.

SEMESTER–VI

Course Code	: DSE-3-BCA-603
Course Title	: Introduction to Machine Learning
Nature of the Course	: MAJ
Total Credit assigned	: 4
Component of the Course	: Lecture: 2 Practical: 2 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 40 In Semester: 12

Course Objectives:

- CO1. To introduce the fundamental concepts, types, and workflow of machine learning, along with model evaluation techniques.
- CO2. To provide practical understanding of supervised and unsupervised learning algorithms, including regression, classification, clustering, and their real-world applications.

Course Content:

Unit	Content	Marks	L	P	T
I	Introduction to Machine Learning <i>Introduction:</i> Definition, History and Application of Machine Learning, <i>Types of Machine Learning:</i> Supervised, Unsupervised, Semi-Supervised, and Reinforcement Learning. Labelled and Unlabelled Dataset. <i>Supervised Learning Tasks:</i> Regression vs. Classification, <i>Learning Framework:</i> Training, Validation and Testing of ML models. <i>Performance Evaluation Parameters:</i> Confusion matrix, Accuracy, Precision, Recall, F1Score, and AUC.	13	15	18	0
II	Supervised Learning and Unsupervised Learning <i>Regression:</i> Linear and non-linear Regression, Logistic Regression. <i>Classification:</i> Naïve Bayes, K-Nearest Neighbors, Decision Trees. <i>Linear model:</i> Introduction to Artificial Neural Networks, Perceptron Learning Algorithm, Single Layer Perceptron, Introduction to Support Vector Machine for linearly separable data. <i>Clustering:</i> K-Means, Hierarchical Clustering, DBSCAN, Clustering Validation Measures. <i>ML Applications:</i> Ethical Considerations in Machine Learning, Case study and Real-world Applications.	15	15	22	0
		28	30	40	

Mode of In-Semester assessment (12 Marks):

- A. Sessional Activity
- B. Sessional Tests: (two Sessional Tests)

Sessional Activities:

- **ML in Real Life**– Group discussion on ML applications (e.g., Netflix, fraud detection).
- **Dataset Activity**– Identify labelled vs unlabelled datasets and match them with ML types.
- **Mini Project: Regression & Classification**– Apply linear/logistic regression and one classifier (e.g., KNN) on real data using Python/Colab.
- **Clustering Demo**– Perform K-Means clustering on a small dataset and interpret results.
- **Model Evaluation Task**– Given confusion matrix data, compute accuracy, precision, recall, F1-score.

Suggested list of LAB Assignments (not limited to):

1. Implement linear regression on a dataset and visualize the regression line.
2. Implement logistic regression on a binary classification dataset and plot the decision boundary.
3. Implement and evaluate the performance of Decision tree ID3/Cart classifier for any given dataset.
4. Implement and evaluate the performance of the Naive Bayes Classifier on a given dataset.
5. Build and evaluate a random forest classifier using a numerical dataset.
6. Implement a support vector machine for linearly separable classes and visualize the margins and decision boundary.
7. Implement K-Means clustering on a point dataset and visualize and evaluate the clusters.
8. Implement hierarchical clustering on a dataset and plot the dendrogram.
9. Implement DBSCAN clustering on a dataset and visualize and evaluate the clusters.
10. Perform Principal Components Analysis (PCA) and apply any one or more classifiers to show the performance variation with or without feature reduction.
11. Build a single layer perceptron model to classify AND, OR, and XOR problems (may use TensorFlow/Keras) and visualize their decision boundaries. Also evaluate its performance.
12. Demonstrate the concept of boosting using the AdaBoost algorithm.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Explain types of machine learning, differentiate between regression and classification, and evaluate models using standard performance metrics.
- CLO-2. Apply ML algorithms like regression, classification, and clustering, and analyze case studies while understanding ethical implications.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁					
Conceptual Knowledge	CO ₁	CO ₂	CO ₂	CO ₂		
Procedural Knowledge		CO ₁	CO ₂	CO ₁		
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3	1	3	3	1	1	2	1	2	1
CLO2	3	1	3	3	1	1	3	1	2	1
Average Contribution	3	1	3	3	1.0	1	2.5	1	2	1.0

(3=High, 2= Moderate 1= Low)

Text Books:

1. Vijayalakshmi, M., & Sridhar, S. (2021). Machine Learning.
2. Kalita, J. K., Bhattacharyya, D. K., & Roy, S. (2023). *Fundamentals of data science: theory and practice*. Elsevier.

Reference Books:

1. Flach, P. (2012). *Machine learning: the art and science of algorithms that make sense of data*. Cambridge university press.
2. Duda, R. O., & Hart, P. E. (2006). *Pattern classification*. John Wiley & Sons..
3. Haykin, S. (2009). *Neural networks and learning machines, 3/E*. Pearson Education India..
4. Chollet, F., & Chollet, F. (2021). *Deep learning with Python*. Simon and Schuster..
5. Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning* (Vol. 4, No. 4, p. 738). New York: springer.
6. Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). Deep learning, 1 MIT press. deeplearningbook.org/contents/numerical.htm (02.07. 2023)..
7. Géron, A. (2017). Hands-On machine learning with Scikit-Learn and TensorFlow: concepts, tools. and *Techniques to Build Intelligent Systems..*

SEMESTER–VI

MAJ-4-CSC-604: Design and Analysis of Algorithms

Course Code	: MAJ-4-CSC-604
Course Title	: Design and Analysis of Algorithms
Nature of the Course	: MAJOR
Total Credit assigned	: 4
Component of the Course	: Lecture: 4 Practical: 0 Tutorial: 0
Distribution of Marks	: End Semester: 56 Practical: 0 In Semester: 24

Course Objectives:

- CO1. To introduce the foundational concepts of algorithm design and performance evaluation using time and space complexity, and asymptotic notations.
- CO2. To study the principles of divide & conquer and greedy algorithm design paradigms, along with their real-world applications and performance analysis.
- CO3. To understand the dynamic programming approach and explore fundamental graph algorithms for traversal and optimization problems.
- CO4. To explore algorithmic problem-solving using backtracking and understand the theoretical limitations of algorithms through complexity classes.

Course Content:

Unit	Content	Marks	L	P	T
I	Introduction: - What is an algorithm? Design and performance analysis of algorithms, time complexity, space complexity. - Asymptotic notations (O, Ω, Θ) to measure growth of a function and application to measure complexity of algorithms. - Analysis of sequential search, bubble sort, selection sort, insertion sort, matrix multiplication. Recursion: Basic concept. Analysis of recursive algorithms, Master's theorem	13	14	0	0
II	The Divide & Conquer Design Technique: - The general concept. Binary search, finding the maximum and minimum, merge sort, quick sort. Best and worst case analysis for the mentioned algorithms. Strassen's matrix multiplication. Lower bound for comparison-based sorting. The Greedy Design Technique: - The general concept. Applications to general Knapsack problem, finding minimum weight spanning trees: Prim's and Kruskal's algorithms, Dijkstra's algorithm for finding single source shortest paths problem	16	17	0	0
III	The Dynamic Programming Design Technique: The general concept. Computation of Fibonacci series and Binomial coefficients, all pair shortest paths problem (Floyd-	14	15	0	0

	Warshall's algorithm), 0/1 Knapsack problem. Algorithms on Graphs: Breadth First Search, Depth First Search, finding connected components, depth first search of a directed graph, topological sorting.				
IV	Limitations of Algorithmic Power: - Backtracking Method: n-Queen problem; sum of subsets problem/Hamiltonian circuit - Problem /vertex cover problem. - Computational Intractability: Overview of non-deterministic algorithms, P, NP, NP-Complete and NP-hard problems.	13	14	0	0
		56	60	0	0

Mode of In-Semester assessment (24 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional Activities:

- Algorithm Complexity Calculation:** Analyze the time and space complexity of basic algorithms using asymptotic notations.
- Sorting Algorithm Implementation:** Implement and test bubble sort, insertion sort, and measure their performance.
- Divide and Conquer Exercise:** Implement merge sort and quick sort, then compare their efficiency.
- Greedy Algorithm Application:** Solve the knapsack problem using a greedy approach and analyze its efficiency.

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Analyze basic algorithms using asymptotic notations and apply complexity analysis techniques to recursive and iterative algorithms.
- CLO-2. Implement and evaluate divide & conquer and greedy algorithms such as merge sort, quick sort, Kruskal's, and Dijkstra's algorithms.
- CLO-3. Solve optimization problems using dynamic programming and apply graph traversal techniques like BFS and DFS for various applications.
- CLO-4. Implement backtracking algorithms and classify problems based on computational complexity into P, NP, NP-Complete, and NP-Hard categories.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁					
Conceptual Knowledge		CO ₁ , CO ₂ , CO ₃ , CO ₄	CO ₂ , CO ₃ , CO ₄	CO ₁ , CO ₂ , CO ₃ , CO ₄		
Procedural Knowledge			CO ₂ , CO ₃ , CO ₄	CO ₄		
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3		3	3			2	1	2	1
CLO2	3		3	3			1	2	2	1
CLO3	3		3	3			2	2	3	1
CLO4	3		3	3			2	2	3	1
Average Contribution	3		3	3			1.75	1.75	2.5	1

(3=High, 2= Moderate 1= Low)

Text Books

1. Cormen, T. H., Leiserson, C. E., & Rivest, R. L. (2009). Stein, Clifford Introduction to Algorithms..
2. Levitin, A. (2008). *Introduction to design and analysis of algorithms*, 2/E. Pearson Education India.
3. Kleinberg, J., & Tardos, E. (2006). *Algorithm design*. Pearson Education India.

SEMESTER–VI

DSE-3-CSC-601: Android Programming using JAVA/Cotlin

Course Code	: DSE-3-CSC-601
Course Title	: Android Programming using JAVA/Cotlin
Nature of the Course	: DSE
Total Credit assigned	: 3
Component of the Course	: Lecture: 2 Practical: 1 Tutorial: 0
Distribution of Marks	: End Semester: 28 Practical: 20 In Semester: 12

Course Objectives:

- CO-1. To introduce students to Android development concepts, Android architecture, and the setup of a development environment including Android Studio.
- CO-2. To familiarize students with Android UI components, layouts, and the implementation of user interaction through events and custom views.
- CO-3. To explain the use of Activities, Fragments, and Intents for managing user navigation and modularizing user interfaces.
- CO-4. To enable students to integrate multimedia (audio, video), messaging (SMS, email), and location-based features into Android apps.
- CO-5. To provide knowledge of data storage using SQLite, networking using RESTful APIs, and the process of testing and publishing Android applications.

Course Content:

Unit	Content	Marks	L	P	T
I	Introduction to Android and Development Environment - History, Features, and Architecture of Android - Android Components: Activities, Services, Broadcast Receivers, Content Providers - Environment Setup: Android Studio, SDK Manager, Emulator - Creating Hello World Application - APK Structure, Organizing Resources	6	6	6	0
II	UI Components and User Interaction - Layouts: LinearLayout, RelativeLayout, ConstraintLayout - Views: TextView, Button, EditText, RadioGroup, ToggleButton, CheckBox, etc. - Event Handling, Menus, Styles & Themes, Alert Dialogs - Custom Components, Drag & Drop	6	6	6	0
III	Fragments, Intents, and Navigation - Activity Lifecycle, Fragment Lifecycle - Intents: Explicit, Implicit, Filters - Navigation between Activities and Fragments - UI Navigation Patterns	6	6	6	0
IV	Multimedia, Messaging & Location Services - Media Player, Audio Manager, Jetplayer, Camera Access - Sending SMS and Email	5	6	6	0

	• Location-Based Services and Google Maps • Push Notifications				
V	Storage, Database, Networking and Publishing • Internal Storage, SharedPreferences, Clipboard • SQLite Database, SQLiteOpenHelper, CRUD operations • Networking: HTTP, JSON using Volley/Retrofit • Publishing Application: Signed APKs, Play Store • Testing, UI Design Patterns, WebView	5	6	6	
		28	30	30	0

Mode of In-Semester assessment (12 Marks):

A. Sessional Activity

B. Sessional Tests: (two Sessional Tests)

Sessional Activities:

1. Weekly Lab Work
 - Students will complete one lab exercise each week.
 - The lab topics will match the theory topics (like making layouts, using buttons, connecting screens, using GPS, etc.).
 - Students will write code and run their apps using Android Studio.
2. Mini Project (Mid-Semester)
 - Students will work alone or in pairs to create a small Android app.
 - The app must have at least two screens and save some data (for example: To-do list, notes app, or calculator).
 - This project helps students apply what they have learned in Units I to III.
3. Code Demo and Peer Review
 - Students will show their working apps to classmates or the teacher.
 - Classmates can give suggestions to improve.
 - This will help in learning from each other's mistakes and ideas.
4. Quizzes and Viva
 - Short quizzes will be given after every few units.
 - There may also be a short viva (oral test) in the lab to check understanding.
 - These help students revise topics regularly.

Suggested list of LAB Assignments (not limited to):

1. Install Android Studio; Create Hello World app
2. Explore Activity lifecycle with Toast logs
3. Create app with multiple Activities using explicit and implicit intents
4. Implement Fragment-based UI for tablets
5. Design layout with ConstraintLayout and custom themes
6. Use widgets like Button, RadioButton, CheckBox
7. Create and query a SQLite database
8. Build an app with local file storage and SharedPreferences
9. Capture and display image from camera; play audio
10. Use Retrofit/Volley to fetch JSON and display in RecyclerView
11. Display user location using Google Maps API
12. Build, test, and export APK; prepare Play Store assets

Expected Course Learning Outcomes:

By the end of the course, students will be able to:

- CLO-1. Describe Android architecture and development tools, and build a basic Android application using Android Studio.
- CLO-2. Design and implement user interfaces using layouts, views, themes, and handle various user events.
- CLO-3. Develop applications using multiple activities, fragments, and intents for seamless navigation and modular design.
- CLO-4. Integrate multimedia features, send SMS and emails, and implement location-based services using Google Maps and notifications.
- CLO-5. Manage local storage using SQLite and SharedPreferences, implement HTTP networking using JSON, and build, test, and publish Android apps to the Play Store.

Cognitive Map of Course Outcomes based on Revised Bloom's Taxonomy

Knowledge Dimension	Remember	Understand	Apply	Analyse	Evaluate	Create
Factual Knowledge	CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅					
Conceptual Knowledge		CO ₁ , CO ₂ , CO ₃ , CO ₄ , CO ₅	CO ₂ , CO ₃ , CO ₄ , CO ₅	CO ₃ , CO ₄ , CO ₅		
Procedural Knowledge			CO ₃ , CO ₄ , CO ₅	CO ₄ , CO ₅		
Metacognitive Knowledge						

Mapping of PSO with CLO

CLO / PSO	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PSO7	PSO8	PSO9	PSO10
CLO1	3		1	2		2	2	1	2	
CLO2	3		3	3		2	2	2	2	
CLO3	3		2	3		2	2	2	2	
CLO4	2		2	3	2	2	1	2	2	
CLO5	3		2	3	2	2	2	2	2	2
Average Contribution	2.8		2	2.8	0.8	2.0	1.8	1.8	2.0	0.4

(3=High, 2= Moderate 1= Low)

Text Books:

1. Nilsson, N. J. (1998). *Artificial intelligence: a new synthesis*. Elsevier.
2. Patterson, D. (1990). *Introduction to artificial intelligence and expert systems*. Prentice-Hall, Inc..

Reference Books:

1. Laurence, P.-O., Hinchman-Dominguez, A., & Dunn, M. (2021). *Programming Android with Kotlin: Achieving structured concurrency with coroutines* (Grayscale Indian ed.). Shroff Publishers.
2. Kumar, P. (2022). *Building Android projects with Kotlin*. BPB Publications.
3. Meike, G. B. (2012). *Programming Android: Java programming for the new generation of mobile devices* (2nd ed.). O'Reilly Media.